



Fakultät Informatik

Evaluation von Contract Testing in Continuous Integration Pipelines zur Sicherstellung der Interoperabilität in einer Microservice-Architektur

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Jonas Lang

Matrikelnummer 363 0314

Erstgutachter: Prof. Dr.-Ing. Matthias Meitner

Zweitgutachter: Prof. Dr. Ronald Petrlic

Betreuer: Sebastian Foersch

Unternehmen: msg systems ag

© 2025

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Hinweis: Diese Erklärung ist mit Originalunterschrift (nicht gescannt) in das Papierexemplar der Abschlussarbeit fest einzubinden. Eine Spiralbindung ist nicht zulässig. Das digitale Exemplar enthält einen Scan der Erklärung mit Unterschrift.

Prüfungsrechtliche Erklärung der/des Studierenden

Angaben des bzw. der Studierenden:

Name: Lang Vorname: Jonas Matrikel-Nr.: 3630314

Fakultät: Informatik Studiengang: Informatik

Semester: SoSe 2025

Titel der Abschlussarbeit:

Evaluation von Contract Testing in Continuous Integration Pipelines zur Sicherstellung der Interoperabilität in einer Microservice-Architektur

Ich versichere, dass ich die Arbeit selbständig verfasst, nicht anderweitig für Prüfungszwecke vorgelegt, alle benutzten Quellen und Hilfsmittel angegeben sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe. Außerdem versichere ich, dass der Hauptteil des Papierexemplares und des digitalen Exemplares identisch sind. Das digitale Exemplar enthält, falls gefordert, lediglich weitere Anlagen.

Altdorf, den 25.06.2025

Ort, Datum, Unterschrift Studierende/Studierender

Erklärung der/des Studierenden zur Veröffentlichung der vorstehend bezeichneten Abschlussarbeit

Die Entscheidung über die vollständige oder auszugsweise Veröffentlichung der Abschlussarbeit liegt grundsätzlich erst einmal allein in der Zuständigkeit der/des studentischen Verfasserin/Verfassers. Nach dem Urheberrechtsgesetz (UrhG) erwirbt die Verfasserin/der Verfasser einer Abschlussarbeit mit Anfertigung ihrer/seiner Arbeit das alleinige Urheberrecht und grundsätzlich auch die hieraus resultierenden Nutzungsrechte wie z.B. Erstveröffentlichung (§ 12 UrhG), Verbreitung (§ 17 UrhG), Vervielfältigung (§ 16 UrhG), Online-Nutzung usw., also alle Rechte, die die nicht-kommerzielle oder kommerzielle Verwertung betreffen.

Die Hochschule und deren Beschäftigte werden Abschlussarbeiten oder Teile davon nicht ohne Zustimmung der/des studentischen Verfasserin/Verfassers veröffentlichen, insbesondere nicht öffentlich zugänglich in die Bibliothek der Hochschule einstellen.

Hiermit ☒ genehmige ich, wenn und soweit keine entgegenstehenden Vereinbarungen mit Dritten getroffen worden sind,

☐ genehmige ich nicht.

dass die oben genannte Abschlussarbeit durch die Technische Hochschule Nürnberg Georg Simon Ohm, ggf. nach Ablauf einer mittels eines auf der Abschlussarbeit aufgebrachten Sperrvermerks kenntlich gemachten Sperrfrist

von 0 Jahren (0 - 5 Jahren ab Datum der Abgabe der Arbeit),

der Öffentlichkeit zugänglich gemacht wird. Im Falle der Genehmigung erfolgt diese unwiderruflich; hierzu wird der Abschlussarbeit ein Exemplar im digitalisierten PDF-Format an die Betreuer übermittelt. Bestimmungen der jeweils geltenden Studien- und Prüfungsordnung über Art und Umfang der im Rahmen der Arbeit abzugebenden Exemplare und Materialien werden hierdurch nicht berührt.

Altdorf, den 25.06.2025

Ort, Datum, Unterschrift Studierende/Studierender

Datenschutz: Die Antragstellung ist regelmäßig mit der Speicherung und Verarbeitung der von Ihnen mitgeteilten Daten durch die Technische Hochschule Nürnberg Georg Simon Ohm verbunden. Weitere Informationen zum Umgang der Technischen Hochschule Nürnberg mit Ihren personenbezogenen Daten sind unter nachfolgendem Link abrufbar: <https://www.th-nuernberg.de/datenschutz/>

Kurzdarstellung

Entwicklungsteams setzen zunehmend auf Microservice-Architekturen, um die Flexibilität und Skalierbarkeit ihrer Anwendungen zu erhöhen. Dabei ist aufgrund komplexerer Entwicklungsprozesse und kürzerer Release-Zyklen die Sicherstellung der Softwarequalität durch regelmäßige Softwaretest eine zentrale Herausforderung. Klassische Testmethoden wie Integrationstests stoßen hierbei an ihre Grenzen, da sie oft zeitintensiv und fehleranfällig sind. Diese Arbeit hat das Ziel die Eignung von Contract Testing zur Verbesserung der Schnittstelleninteroperabilität, Testeffizienz und Fehlererkennung systematisch zu untersuchen.

Zunächst erfolgte eine vergleichende Bewertung von fünf relevanten Contract-Testing-Frameworks (**Pact**, **Spring Cloud Contract**, **Specmatic**, **Karate**, **ContractCase**) anhand definierter technischer, organisatorischer und funktionaler Kriterien. Im Anschluss wurde das Framework **Pact** für die prototypische Implementierung von Contract Testing ausgewählt und in der realen Microservice-Anwendung *msg.Sensorik* umgesetzt. Die Implementierung wurde hinsichtlich ihrer Fähigkeit zur Fehlererkennung, ihrer Effizienz im Vergleich zu klassischen Integrationstests sowie ihres Beitrags zur Reduktion der Fehlerkosten evaluiert. Die Ergebnisse zeigen, dass Contract Testing, insbesondere in CI/CD-gestützten Entwicklungsprozessen, als ergänzende Testmethode signifikant zur frühzeitigen Fehlererkennung und somit zur Kosteneinsparung bei der Entwicklung beitragen kann.

Inhaltsverzeichnis

1 Einführung	1
1.1 Problemstellung und Motivation	1
1.2 Zielsetzung der Arbeit	2
1.3 Aufbau der Arbeit	3
2 Grundlagen	5
2.1 Microservice-Architektur	5
2.2 Arten des Softwaretests	6
2.3 Testautomatisierung	8
2.4 Contract Testing	9
3 Contract Testing Frameworks	17
3.1 Übersicht verfügbarer Frameworks	17
3.2 Auswahlkriterien	19
3.3 Vergleich der Frameworks	23
4 Prototypische Implementierung	31
4.1 Anwendungsumgebung	31
4.2 Technische Umsetzung	33
4.3 Aufgetretene Herausforderungen	39
5 Evaluation der Ergebnisse	43
5.1 Bewertung der Fehlererkennung	43
5.2 Bewertung der Testeffizienz	45
6 Fazit und Ausblick	49
6.1 Fazit	49
6.2 Ausblick	51
A Weiterführende Informationen	53
A.1 Komplexitätsunterschied der untersuchten Testpipelines	53
A.2 Tatsächliche Messwerte der Testpipeline-Ausführungszeiten	54
Abbildungsverzeichnis	55
Tabellenverzeichnis	57

Verzeichnis der Listings	59
Glossar	61
Akronyme	63
Literatur	65

Kapitel 1

Einführung

Microservice-Architekturen haben sich in den letzten Jahren als architektonisches Paradigma für moderne Softwaresysteme etabliert [18]. Sie bieten eine Vielzahl von Vorteilen, z. B. eine erhöhte Skalierbarkeit, modulare Wartbarkeit sowie die Möglichkeit, einzelne Dienste unabhängig voneinander zu entwickeln und zu deployen. Gleichzeitig bringen sie jedoch neue Herausforderungen mit sich – insbesondere im Bereich der Qualitätssicherung und der Sicherstellung der Interoperabilität zwischen den einzelnen Services [30].

1.1 Problemstellung und Motivation

Eine zentrale Problemstellung liegt darin, dass Änderungen an einzelnen Microservices potenziell unerwartete Seiteneffekte auf abhängige Services haben können [8]. Klassische Testmethoden wie Systemtests und Integrationstests sind zwar in der Lage, solche Probleme aufzudecken, jedoch sind diese oft mit hohen Kosten, langen Ausführungszeiten und einer erhöhten Komplexität in der Wartung verbunden. Dies macht sie vornehmlich in regelmäßig ausgeführten Continuous Integration (CI)/Continuous Delivery (CD)-Pipelines schwer handhabbar und ineffizient [43]. Eine CI/CD-Pipeline ist ein automatisierter Prozess, der es ermöglicht, Softwareänderungen kontinuierlich zu integrieren, zu testen und bereitzustellen und daher im Bereich der Softwareentwicklung weit verbreitet [15].

Contract Testing hat sich als eine potenziell leistungsfähige Testmethode etabliert. Es ermöglicht eine gezielte Validierung von Service-Schnittstellen, indem Contracts zwischen Consumer- und Provider-Services definiert und automatisch überprüft werden [11]. Allerdings gibt es bisher nur begrenzte wissenschaftliche Untersuchungen zur praktischen Umsetzung und Wirksamkeit dieser Methodik.

Daher ist es essenziell, eine detaillierte Untersuchung zur Anwendung von Contract Testing in CI/CD-Pipelines durchzuführen, um herauszufinden, ob und inwieweit diese Testmethode eine nachhaltige Verbesserung der Interoperabilität in einer Microservice-Architektur gewährleisten kann.

1.2 Zielsetzung der Arbeit

Ziel dieser Bachelorarbeit ist es, Contract Testing im Kontext von CI/CD-Pipelines systematisch zu analysieren, praktisch zu implementieren und hinsichtlich seines Nutzens zur Sicherstellung der Interoperabilität in einer Microservice-Architektur zu evaluieren. Im Zentrum steht die prototypische Integration von Contract Testing in die reale Microservice-Anwendung *msg.Sensorik* aus dem Unternehmenskontext der *msg systems ag*¹.

Auf dieser Basis sollen folgende Fragestellungen beantwortet werden:

- Welche konkreten Vorteile ergeben sich durch die Einführung von Contract Testing im Vergleich zu der ausschließlichen Verwendung von klassischen Softwaretests?
- Inwieweit trägt Contract Testing zur frühzeitigen Fehlererkennung und Effizienzsteigerung in CI/CD-Pipelines bei?
- Welche Contract-Testing-Frameworks sind für die Integration in bestehende Anwendungsumgebungen geeignet und welche Kriterien sind bei der Auswahl zu berücksichtigen?
- Welche Herausforderungen treten bei der praktischen Umsetzung auf und wie lassen sich diese bewältigen?
- Welche Handlungsempfehlungen lassen sich daraus für andere Projekte ableiten?

Ein besonderes Augenmerk liegt dabei auf der Bewertung anhand quantitativer und qualitativer Kriterien – beispielsweise Fehlererkennungsrate, Testdurchlaufzeit und Komplexität der Testinfrastruktur. Ziel ist es, eine belastbare Grundlage für den praktischen Einsatz von Contract Testing in produktionsnahen Umgebungen zu schaffen.

Dazu wird zunächst eine vergleichende Analyse von Contract-Testing-Frameworks anhand festgelegter technischer, organisatorischer und funktionaler Kriterien durchgeführt. Daraufhin wird ein Framework für die prototypische Umsetzung von Contract Testing ausgewählt, das anschließend in die reale Microservice-Anwendung *msg.Sensorik* implementiert wird. Die Evaluation der Implementierung konzentriert sich auf ihre Fehlererkennungsfähigkeit, ihre Effizienz gegenüber klassischen Integrationstests sowie ihren Einfluss auf die Reduktion von Fehlerkosten.

¹<https://www.msg.group>

1.3 Aufbau der Arbeit

Die vorliegende Arbeit ist in sieben Kapitel gegliedert, die systematisch von den theoretischen Grundlagen bis zur praktischen Umsetzung und den daraus resultierenden Erkenntnissen führen.

Kapitel 2 vermittelt das notwendige Hintergrundwissen. Es behandelt zentrale Konzepte wie Microservice-Architekturen, Continuous Integration sowie gängige Teststrategien und stellt das Prinzip der Contract Tests im Detail vor.

Im darauffolgenden Kapitel 3 werden verschiedene Contract-Testing-Frameworks untersucht. Anhand definierter Kriterien erfolgt ein systematischer Vergleich, auf dessen Grundlage ein geeignetes Werkzeug für die Implementierung ausgewählt wird.

Kapitel 4 widmet sich der praktischen Umsetzung. Es beschreibt die konkrete Anwendungsumgebung, die technischen Voraussetzungen sowie die schrittweise Integration der Contract Tests in eine bestehende Microservice-Infrastruktur und deren CI/CD-Umgebung.

Die Wirksamkeit dieser Integration wird in Kapitel 5 analysiert. Hier werden die Ergebnisse anhand quantitativer Messungen und qualitativer Beobachtungen bewertet – insbesondere in Bezug auf Fehlererkennung, Testeffizienz und potenzielle Einsparungen gemäß der *Cost-of-Failure*-Kurve.

Abschließend wird in Kapitel 6 ein Fazit gezogen. Es fasst die zentralen Erkenntnisse der Arbeit zusammen, diskutiert bestehende Einschränkungen und zeigt mögliche Perspektiven für weiterführende Forschung oder den produktiven Einsatz in anderen Projekten auf.

Kapitel 2

Grundlagen

Bevor die praktische Umsetzung von Contract Testing erfolgen kann, ist ein fundiertes Verständnis der zugrunde liegenden Konzepte notwendig. Dieses Kapitel stellt die theoretischen Grundlagen bereit, die für das Verständnis der weiteren Arbeit zentral sind. Zunächst wird die Microservice-Architektur als modernes Software-Entwurfsmuster erläutert, da Contract Testing in diesem Kontext besonders relevant ist und sie die Basis für die Anwendungsumgebung der prototypischen Implementierung in Kapitel 4 bildet.

2.1 Microservice-Architektur

Die Microservice-Architektur ist ein Architekturstil in der Softwareentwicklung, bei dem eine Anwendung als eine Sammlung unabhängiger, kleiner Dienste aufgebaut wird. Jeder dieser Dienste ist eigenständig und erfüllt eine spezifische Anwendungsaufgabe. Diese Architektur hat sich als Alternative zu einer monolithischen Architektur etabliert, bei der alle Funktionalitäten einer Anwendung in einer einzigen, großen Codebasis zusammengefasst werden [18].

Ein wesentliches Merkmal der Microservice-Architektur ist die Unabhängigkeit der Dienste. Jeder Microservice kann eigenständig entwickelt, getestet, bereitgestellt und skaliert werden. Das Zusammenspiel zwischen den Diensten wird durch klar definierte Application Programming Interfaces (APIs) sichergestellt [41]. Zudem wird die funktionale Trennung betont, da jeder Service einen klar definierten Anwendungsteil abdeckt und von einem eigenen Entwicklungsteam verwaltet werden kann. Diese Struktur ermöglicht es, unterschiedliche Technologien oder Programmiersprachen für verschiedene Services einzusetzen, wodurch eine technologische Vielfalt innerhalb eines Systems entsteht [13]. So entstehen mehrere kleine Anwendungen, die überschaubarer und einfacher zu warten sind als ein großer Technologie-Stack, wie z. B. in einer monolithischen Architektur [11], bei der alle Komponenten einer Anwendung in einer unteilbaren Einheit integriert sind. Diese Eigenschaften machen die Microservice-Architektur besonders attraktiv für moderne, dynamische Softwaresysteme.

Aufbauend auf diesen Grundlagen lassen sich auch die spezifischen Vorteile und Herausforderungen dieser Architektur ausmachen.

Die Microservice-Architektur bietet zahlreiche Vorteile. Sie ermöglicht eine hohe Flexibilität, da Änderungen an einem Service unabhängig von anderen Anwendungsteilen vorgenommen werden können [18]. Die Fehlertoleranz wird verbessert, weil ein Fehler in einem Microservice nicht zwangsläufig die gesamte Anwendung beeinträchtigt. Durch die gezielte Bereitstellung von Ressourcen für bestimmte Dienste kann die Skalierbarkeit optimiert werden, indem einzelne Services bei Bedarf optimiert werden, ohne die restliche Anwendung zu beeinflussen [30]. Zudem erleichtert die überschaubare Größe einzelner Microservices deren Wartung, da der Code besser verständlich und weniger komplex ist [31].

Trotz der vielen Vorteile bringt die Microservice-Architektur auch Herausforderungen mit sich. Die Kommunikation zwischen den einzelnen Diensten erfordert robuste Schnittstellen und ein zuverlässiges API-Management, um eine reibungslose Interaktion zu gewährleisten. Die Verwaltung von Nutzdaten kann komplex sein, da die Konsistenz zwischen verschiedenen Microservices sichergestellt werden muss, d. h. die Schnittstellen der einzelnen Dienste müssen aufeinander abgestimmt sein. Darüber hinaus erfordert die Vielzahl kleiner Dienste leistungsfähige Deployment- und Monitoring-Tools, um deren Verwaltung effizient zu gestalten [30].

Die Microservice-Architektur in ihrer Gesamtheit bildet die Grundlage für moderne verteilte Systeme und spielt eine zentrale Rolle für Cloud-Umgebungen, weil sie eine flexible und skalierbare Infrastruktur bietet. Cloud-Umgebungen ermöglichen den Betrieb und die Bereitstellung von IT-Ressourcen unabhängig von physischer Hardware und bieten dabei ein hohes Maß an Flexibilität [17]. Diese Architektur verbreitet sich daher zunehmend und wird in Zukunft immer häufiger eingesetzt werden [41]. Da bei dieser Architektur die Kommunikation zwischen den einzelnen Diensten im Vordergrund steht, ist eine zuverlässige Validierung der Schnittstellen essenziell. Diese kann durch verschiedene Testarten erfolgen, die im Folgenden erläutert werden.

2.2 Arten des Softwaretests

Um die Qualität und Funktionsfähigkeit von Software und ihren Schnittstellen sicherzustellen, können verschiedene Arten von Softwaretests zum Einsatz kommen. Diese lassen sich nach ihrem Fokus und ihrer Testtiefe kategorisieren. Die Hierarchie der verschiedenen klassischen Teststufen wird in Abbildung 2.1 dargestellt. Die unterschiedlichen Testarten werden im Anschluss erklärt.

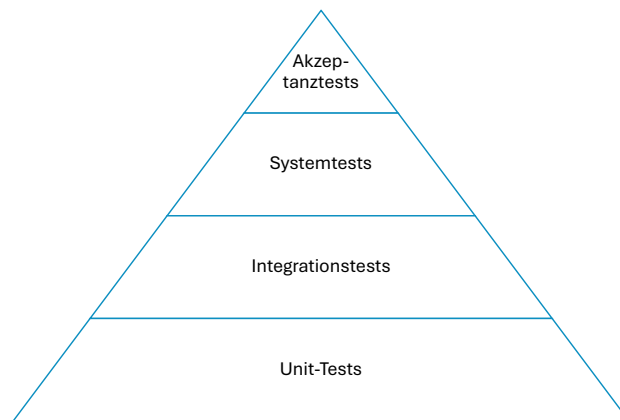


Abbildung 2.1: Testpyramide nach [24]

Unit-Tests (Modultests) überprüfen einzelne Codeeinheiten isoliert von anderen Komponenten. Sie dienen dazu, die korrekte Implementierung kleinster funktionaler Einheiten, wie z. B. Funktionen und Methoden, sicherzustellen. Bei dieser Testart sollten möglichst viele Einheiten getestet werden, um eine hohe Testabdeckung zu erreichen. Wegen der verhältnismäßig geringen Ausführungszeit kann deren Durchführung sehr schnell, beliebig oft und aufwandsarm erfolgen [24]. Diese Tests sind in der Regel automatisiert und werden häufig während der Entwicklung durchgeführt, um Fehler frühzeitig zu erkennen.

Integrationstests hingegen testen das Zusammenspiel mehrerer Module oder Systeme. Sie prüfen, ob die verschiedenen Komponenten korrekt miteinander interagieren. Integrationstests sind wichtig, um sicherzustellen, dass die einzelnen Module nicht nur isoliert, sondern auch in Kombination miteinander funktionieren. Die Testumgebung ist hierbei oft komplexer, da mehrere Module gleichzeitig zur Verfügung stehen müssen, wodurch höhere Laufzeiten und ein erhöhter Ressourcenverbrauch entstehen können [40].

Systemtests betrachten die Anwendung als Ganzes. Mit dieser Art von Tests wird überprüft, ob das gesamte System konform zu den Anforderungen funktioniert. Diese Tests sind in der Regel sehr umfangreich in der Erstellung und Ausführung, da alle Komponenten und deren Interaktionen berücksichtigt werden. Darum sollte man sich auf wenige Testfälle beschränken, die jedoch alle kritischen Aspekte der Anwendung abdecken [24].

Akzeptanztests bilden die höchste Testebene und stellen sicher, dass die Anwendung aus Sicht der Endbenutzer den definierten Anforderungen entspricht. Diese Tests werden häufig manuell und in enger Abstimmung mit den Stakeholdern durchgeführt [43]. Sie sind entscheidend für die Validierung der Software und deren Akzeptanz durch die Benutzer. Aufgrund des hohen Aufwands sind diese Tests nicht Teil von Entwicklungszyklen, sondern werden

in der Regel erst nach der Erreichung von Meilensteinen bzw. nach der Fertigstellung des Systems durchgeführt [24].

Neben diesen klassischen Testarten gibt es auch spezialisierte Testmethoden wie Performance-, Sicherheits- und Usability-Tests. Diese Tests konzentrieren sich auf spezifische Aspekte der Softwarequalität und sind darauf ausgelegt, bestimmte Anforderungen zu validieren. Deshalb sind diese nicht in die Hierarchie der Testpyramide eingeordnet, sondern ergänzen diese in der Praxis.

Um diese verschiedenen Testarten effizient und regelmäßig durchzuführen, insbesondere in agilen Entwicklungsumgebungen, gewinnt die Automatisierung von Tests zunehmend an Bedeutung.

2.3 Testautomatisierung

Mit dem zunehmenden Bedarf an schneller Softwareauslieferung – bei gleichbleibend hoher Qualität – hat sich die Testautomatisierung zu einem zentralen Bestandteil moderner Entwicklungsprozesse entwickelt [15]. Sie beschreibt die kontinuierliche und nicht manuelle Durchführung von Softwaretests, die meist mithilfe spezialisierter Tools und Skripte erfolgt, um den manuellen Testaufwand zu reduzieren, Wiederholbarkeit zu gewährleisten und Fehler frühzeitig im Entwicklungsprozess zu erkennen [31].

Solche Praktiken und Tools werden unter dem Begriff DevOps zusammengefasst. DevOps beschreibt eine kulturelle und technische Bewegung, die darauf abzielt, die Zusammenarbeit zwischen Entwicklungs- (Development) und Betriebsteams (Operation) zu verbessern [51]. Ziel ist es, den gesamten Software-Lebenszyklus zu optimieren, von der Entwicklung über das Testen bis hin zu Deployment und Betrieb [27]. Dazu zählen Aufgaben wie Kollaboration zwischen den Teams, Automatisierung von regelmäßigen Aufgaben, Überwachung von Anwendungsinfrastruktur und Auswertung von Nutzerfeedback [19].

Einige dieser Tätigkeiten werden mithilfe von Continuous Integration automatisiert. Das bedeutet, dass Änderungen am Code kontinuierlich in das gemeinsame Repository integriert werden und dabei automatisierte Bauprozesse der Software und Tests ausgeführt werden, um sicherzustellen, dass der neue Code keine bestehenden Funktionalitäten beeinträchtigt [50]. Durch diese sofortige Rückmeldung können Fehler frühzeitig erkannt und behoben werden, was nicht nur die Codequalität verbessert, sondern auch langfristig Entwicklungskosten senkt. Dies steht im Einklang mit der sogenannten *Cost of Failure*-Kurve, die zeigt, dass die Kosten zur Behebung eines Fehlers exponentiell steigen, je später dieser im Softwareentwicklungsprozess entdeckt wird [4].

2.4 Contract Testing

Continuous Delivery geht noch einen Schritt weiter: Nach der erfolgreichen Integration und dem Bestehen bestimmter Qualitätsziele wird die Anwendung automatisiert für die Auslieferung vorbereitet und bereitgestellt. Dies schließt Prozesse wie das Verpacken, Bereitstellen und Testen der Anwendung in Betriebsumgebungen ein [22]. In der weitergehenden Form erfolgt sogar die automatische Auslieferung der Anwendung in die Produktionsumgebung, was Continuous Deployment genannt wird [12].

Testautomatisierung ist dabei das Rückgrat dieser Abläufe [51]. Sie ermöglicht nicht nur die schnelle und zuverlässige Durchführung von Unit-, Integrations- oder Systemtests, sondern auch die Validierung nichtfunktionaler Anforderungen wie Performance oder Sicherheit [11]. Diese Automatisierungen werden mithilfe von CI/CD-Pipelines orchestriert, die den gesamten Prozess von der Codeänderung bis zur Bereitstellung steuern. Dabei werden die einzelnen Automatisierungsschritte und deren Abfolge definiert und der Auslöser der Durchführung festgelegt [50].

Wie in Abschnitt 2.1 erläutert, ist besonders in Microservice-Architekturen die Testautomatisierung sinnvoll, da hier viele unabhängige Komponenten miteinander interagieren [18]. Ohne automatisierte Tests würde das Risiko fehlerhafter Integrationen oder regressiver Effekte drastisch steigen [50] und damit auch der monetäre Aufwand zur Behebung [4].

Insgesamt trägt Testautomatisierung in Verbindung mit DevOps und CI/CD entscheidend dazu bei, die Qualität, die Sicherheit und die Stabilität moderner Softwareprodukte zu gewährleisten [42]. Im Kontext von Microservices gewinnen automatisierte Tests und insbesondere Contract Tests an Bedeutung, da sie die Kommunikation zwischen Diensten absichern und frühzeitig Fehler in der Interaktion erkennen lassen [11]. Die besondere Testart Contract Testing wird im Folgenden näher erläutert und bildet die Grundlage für den praktischen Teil dieser Arbeit.

2.4 Contract Testing

Contract Testing ist eine Testmethode, die darauf abzielt, die Interoperabilität zwischen verschiedenen Schnittstellen zu gewährleisten. Sie kombiniert die Funktion von API-Integrationstests mit der Zuverlässigkeit und Schnelligkeit von Unit-Tests und kann so Fehler bei der Schnittstellenkommunikation früh im Entwicklungsprozess erkennen [11].

2.4.1 Funktionsweise

Bei der Kommunikation mit APIs sind immer zwei Parteien beteiligt. Zum einen gibt es einen Anbieter (Provider), der die Schnittstelle bereitstellt, und zum anderen ein bis mehrere Verbraucher (Consumer), die diese Schnittstelle nutzen [31]. Wenn ein Consumerdienst eine

Schnittstelle ansprechen möchte, um z. B. Daten zu erhalten oder eine Aktion auszulösen, sendet er eine Anfrage an die API des Providers. Der Provider verarbeitet diese Anfrage, führt eventuell zugehörige Aktionen aus und sendet eine entsprechende Antwort zurück. Auf der Consumer-Seite wird die Antwort nach Erhalt verarbeitet, um die gewünschten Informationen zu extrahieren und zu verwenden. Um diese Verarbeitung zu ermöglichen, trifft der Consumer eine Annahme über die Struktur der zur Anfrage gehörigen Antwort. In einer idealen Welt würde die Antwort immer der Struktur entsprechen, die der Consumer erwartet, doch das kann in der Praxis nicht immer garantiert werden [53]. Wenn die Antwort nicht mit der Annahme übereinstimmt, kann es zu Fehlern kommen. Dies kann zu ungewünschtem Verhalten oder sogar zum Serviceausfall aufseiten des Consumers führen. Contract Testing wird eingesetzt, damit die getroffenen Annahmen tatsächlich eintreffen werden und somit Fehler in der Kommunikation zwischen den Diensten vermieden werden [11].

Bei der Ausführung der Contract Tests wird ein sogenannter „Contract“ definiert – ein Vertrag, der beschreibt, wie die Kommunikation zwischen Consumer und Provider ablaufen soll. Dieser Contract legt fest, welche Anfragen der Consumer an den Provider stellt und welche Antworten er im Gegenzug erwartet. Der entscheidende Vorteil dabei ist, dass beide Seiten unabhängig voneinander testen können, ob sie diesen Contract einhalten [31]. Beim Contract Testing wird der Fokus also nicht auf die vollständige Funktionalität des Systems gelegt, sondern auf die Schnittstelle und die Erwartungen, die damit einhergehen.

In einem Contract werden die Anfragen des Consumers mit den zugehörigen Details, wie dem Hypertext Transfer Protocol (HTTP)-Verb, den Headern usw. festgehalten. Eine stark vereinfachte Anfrage wäre zum Beispiel ein GET-Request an den Pfad `/user/1`. Zu dieser Anfrage wird die zugehörige Antwort definiert, die der Provider liefern soll, wenn der Consumer diese Anfrage stellt. Auch hier werden verschiedene Details festgehalten, wie der HTTP-Statuscode, die Header und der Inhalt der Antwort. Ein Beispiel für eine solche Antwort könnte ein JavaScript Object Notation (JSON)-Objekt sein, das Informationen des Nutzers mit der ID 1 enthält, wie z. B. `{"id": 1, "name": "Max Mustermann"}`. Dieser Contract wird verwendet, um Tests sowohl auf Consumer- als auch auf Provider-Seite durchzuführen [16]. Auf der Consumer-Seite wird überprüft, ob die Anwendung tatsächlich nur die im Contract beschriebenen Anfragen stellt. Auf der Provider-Seite hingegen wird getestet, ob die API die versprochenen Antworten liefert – unabhängig davon, welcher Consumer die Anfrage stellt. Damit wird sichergestellt, dass Änderungen auf einer Seite wie z. B. umbenannte Felder oder veränderte Strukturen nicht unbeabsichtigt zu Fehlern auf der anderen Seite führen [11]. Wenn beispielsweise das Feld `name` beim Provider in `fullName` umbenannt wird und somit der Provider mit `{"id": 1, "fullName": "Max Mustermann"}` antwortet, würden die Contract Tests auf der Provider-Seite fehlschlagen, da die Antwort nicht mehr mit dem Contract konform ist. Daraufhin können unmittelbar Maßnahmen ergriffen werden, um den Contract zu aktualisieren und die Kompatibilität zwischen Consumer und Provider wiederherzustellen [28].

2.4.2 Varianten

Es gibt zwei verschiedene Ansätze Contract Testing aufzubauen, als Consumer-Driven Contract Testing (CDCT) und als Provider-Driven Contract Testing (PDCT) [11].

Consumer-Driven Contract Testing

Wenn der Consumer den Contract erstellt und ihn anschließend mit dem Provider teilt, spricht man von CDCT. Diese Variante ist besonders geeignet für organisationsinterne Anwendungen und für verteilte Software über mehrere Teams hinweg, wie z. B. bei Microservice-Architekturen [41]. Dabei kann der Provider genau auf die Anforderungen des Consumers zugeschnitten werden, da der Consumer die Anfragen im Vorhinein definiert, die er an den Provider stellen möchte [11].

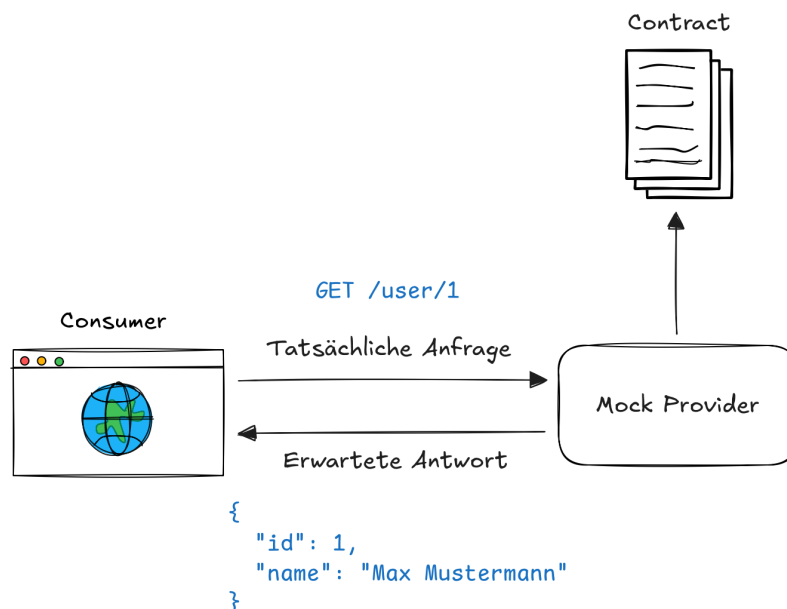


Abbildung 2.2: Ablauf eines Contract Tests auf der Consumer-Seite beim CDCT nach [11]

Ein Contract Test auf der Consumer-Seite beim CDCT, wie in Abbildung 2.2 anhand des oben genannten Beispiels dargestellt, lässt sich in drei Schritte unterteilen:

1. Es wird ein Mock-Provider erstellt, der die API des Providers simuliert, damit der Test vom tatsächlichen Provider unabhängig ist. Dabei wird pro Anfrage des Consumers festgelegt, welche Antwort vom Provider dazu erwartet wird [16].
2. Der Consumer führt die Contract Tests gegen den Mock-Provider aus. Dadurch wird überprüft, ob der Consumer tatsächlich nur die definierten Anfragen stellt und die erwarteten Antworten korrekt verarbeiten kann [11].

3. Wenn alle Contract Tests auf Consumer-Seite erfolgreich absolviert wurden, wird der Contract automatisch generiert, welcher dadurch genau die definierten Interaktionen beinhaltet. Der Contract wird dann so gespeichert, dass er auf der Provider-Seite für deren Contract Tests verwendet werden kann [28]. Die verschiedenen Arten der Speicherung werden in Kapitel 3 näher beschrieben.

Auf der Provider-Seite wird der Contract dann verwendet, um sicherzustellen, dass die API tatsächlich die erwarteten Antworten liefert. Auch der Provider kommuniziert nicht mit dem echten Consumer, sondern mit einem Mock-Consumer, der die Anfragen simuliert, um die Unabhängigkeit der Tests zu gewährleisten [11]. Für jede Anfrage im Contract wird ein Testfall generiert, in dem der Mock-Consumer die Anfrage an die Provider-Anwendung sendet. Der Provider verarbeitet die Anfrage und sendet eine entsprechende Antwort zurück. Der Test überprüft dann, ob die Antwort des Providers mit der im Contract definierten Antwort übereinstimmt, und verifiziert somit, dass die API des Providers den Contract erfüllt [28]. Dieser Ablauf eines Contract Tests auf der Provider-Seite beim CDCT ist in Abbildung 2.3 beispielhaft dargestellt.

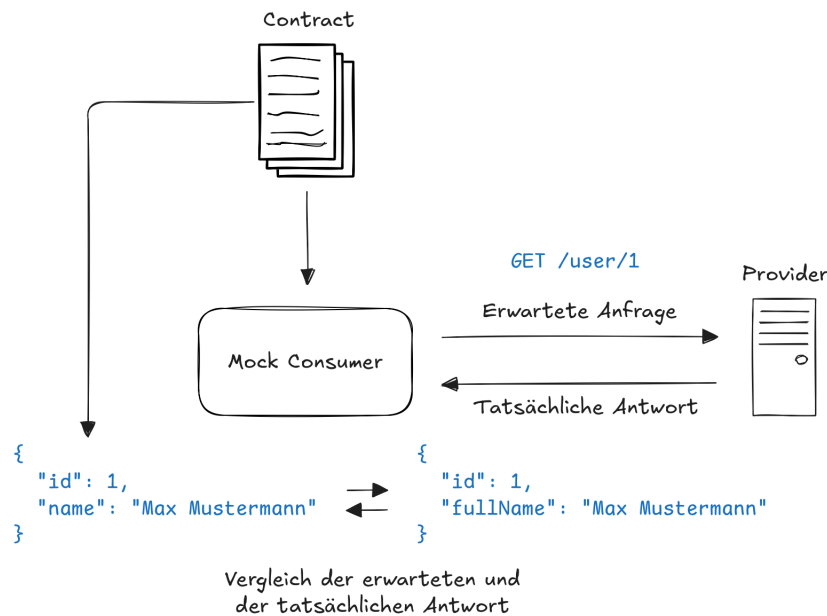


Abbildung 2.3: Ablauf eines Contract Tests auf der Provider-Seite beim CDCT nach [11]

Nun wurde der Contract auf der Consumer-Seite erstellt und auf der Provider-Seite verifiziert. Das bedeutet, dass beide Seiten unabhängig voneinander auch nach Änderungen ihre Contract Tests durchführen und die Services ohne Kompatibilitätsprobleme deployt werden können, wenn sie den Contract einhalten [52]. Allerdings muss für das Deployen jedes

2.4 Contract Testing

Services sichergestellt werden, dass dazu immer ein verifizierter Contract besteht. Im Umkehrschluss bedeutet das aber auch, dass bei neuen Anforderungen des Consumers an die API des Providers auch der Contract aktualisiert und vom Provider neu verifiziert werden muss [11]. Wie dieses Contract Management funktionieren kann, wird in Abschnitt 3.2.8 und Abschnitt 3.2.9 näher erläutert.

Das Verwalten der Contracts ist aufwändig, da es für eine API viele verschiedene Consumer und Versionsstände geben kann [18]. Welche Unterschiede und Varianten es für die Speicherung der Contracts gibt, wird in Abschnitt 3.1 genauer betrachtet.

Wenn die Consumer-driven Variante implementiert wird, hat es den Vorteil, dass der Provider genau die Anforderungen des Consumers erfüllt und keine unbenutzten Funktionalitäten entwickelt bzw. bereitgestellt werden. Da der Consumer genau die Anforderungen stellt, die er benötigt, kann der Provider seine API genau darauf anpassen. Allerdings erfordert das auch eine gute Kommunikation und idealerweise eine geteilte Testinfrastruktur zwischen den beiden Parteien [11].

Provider-Driven Contract Testing

In Szenarien, bei denen z. B. kein Zugriff auf die Test-Pipeline des Providers besteht, kann auf das PDCT zurückgegriffen werden. Dies kann vorkommen, wenn der Provider eine öffentliche API bereitstellt, die von vielen verschiedenen Consumer genutzt wird, oder wenn der Provider und der Consumer von unterschiedlichen Organisationen entwickelt werden [11]. Dabei werden vom Provider und vom Consumer jeweils ein Contract erstellt, die dann automatisiert miteinander verglichen werden. Deshalb wird PDCT auch als bidirektionales Contract Testing bezeichnet [11]. Das bedeutet, dass der Provider einen Contract erstellt, der beschreibt, welche Anfragen er erwartet und welche Antworten er bereitstellt. Damit wird Contract Testing auch bei Kommunikationen mit öffentlichen oder Drittanbieter-APIs ermöglicht, um die Kompatibilität mit diesen sicherzustellen und auf Änderungen dieser reagieren zu können [11]. Auf der Consumer-Seite geschieht die Contract-Erstellung analog zum CDCT. Danach werden die beiden Contracts typischerweise mithilfe eines Brokers verglichen, die sogenannte cross-contract validation. Durch diese Validierung ist die Interoperabilität der beiden Services sichergestellt. Der Broker fungiert dabei als Speicherort, der die Contracts der Consumer und Provider verwaltet und die Validierung durchführt [11]. Wie beim CDCT ist es notwendig, dass beim Deployment-Prozess überprüft wird, ob die entsprechenden Contracts gegenseitig validiert wurden [38]. Der Ablauf beim PDCT ist in Abbildung 2.4 dargestellt.

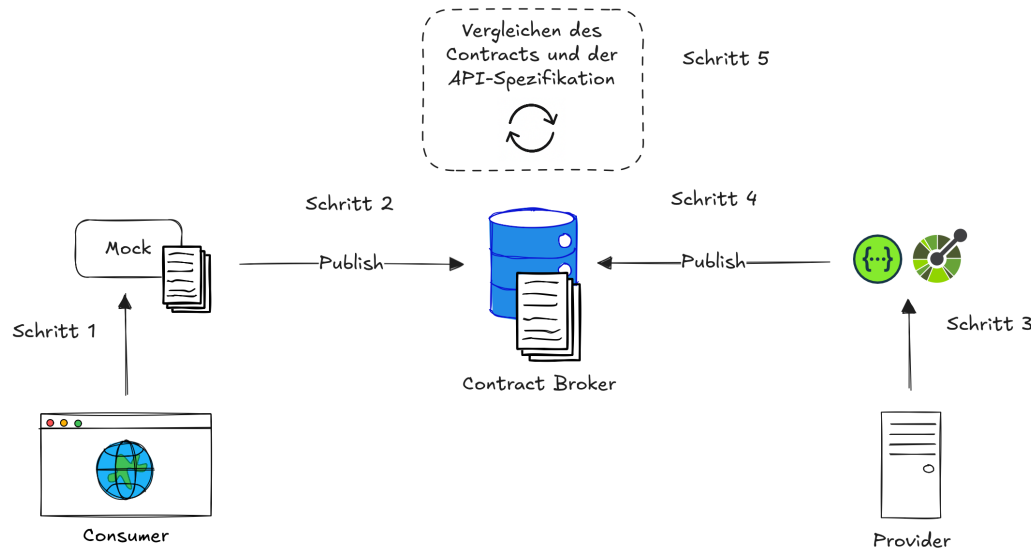


Abbildung 2.4: Ablauf eines Contract Tests beim PDCT nach [11]

Nachdem die grundlegenden Prinzipien sowie die Unterschiede zwischen consumer-driven und provider-driven Contract Testing dargelegt wurden, ist es für eine umfassende Bewertung dieses Testansatzes erforderlich, auch dessen Vor- und Nachteile zu untersuchen. Im folgenden Abschnitt werden daher die wesentlichen Stärken und Schwächen des Contract Testings analysiert, um dessen Nutzenpotenzial und etwaige Limitationen im Kontext moderner Softwarearchitekturen differenziert zu betrachten.

2.4.3 Vor- und Nachteile

Contract Testing hat sich als effektive Methode etabliert, um die Kommunikation über APIs sicherzustellen, und bildet damit die Grundlage für eine verlässliche Integration unterschiedlicher Systeme, vornehmlich in modernen, serviceorientierten Architekturen [31].

Ein wesentlicher Vorteil von Contract Testing besteht in der frühzeitigen Erkennung von Integrationsfehlern. Da die erwarteten Interaktionen bereits während der Entwicklung definiert und automatisiert überprüft werden, lassen sich Abweichungen zwischen Consumer und Provider schnell identifizieren und beheben. Dies führt zu einem schnelleren Feedback für die Entwicklungsteams und reduziert das Risiko, dass Integrationsprobleme erst in späteren Testphasen oder sogar im Produktivbetrieb auftreten [11]. Insbesondere im Rahmen von CI/CD-Pipelines kann Contract Testing dazu beitragen, Fehlerquellen automatisch zu identifizieren, frühzeitig zu eliminieren und die Qualität der Software durch das fehlerfreie Zusammenspiel verschiedener Softwarekomponenten kontinuierlich sicherzustellen [15].

2.4 Contract Testing

Ein weiterer bedeutender Vorteil liegt in der Reduzierung der Abhängigkeit von klassischen Systemtests. Während Systemtests häufig ressourcenintensiv und fehleranfällig sind, ermöglicht Contract Testing eine gezielte Überprüfung der Schnittstellenkompatibilität, ohne dass das gesamte System integriert und getestet werden muss [8]. Dadurch werden die Testzyklen verkürzt und die Entwicklung beschleunigt, da Teams unabhängig voneinander an ihren Komponenten arbeiten können, solange die definierten Contracts eingehalten werden. Dies fördert eine parallele Entwicklung und steigert die Agilität, vornehmlich in großen Projekten mit mehreren Teams [11]. Die Verbesserung der Fehlererkennung und der Testeffizienz werden in Kapitel 5 anhand der prototypischen Implementierung näher untersucht.

Darüber hinaus trägt Contract Testing zu einer verbesserten Dokumentation und Transparenz der Service-Schnittstellen bei. Die Contracts dienen als stets aktuelle Referenz und machen die Erwartungen an die Kommunikation zwischen den Komponenten verbindlich. Dies erleichtert nicht nur die Zusammenarbeit zwischen den beteiligten Teams, sondern unterstützt auch das Anlernen neuer Entwickler, da die Schnittstellenanforderungen klar und nachvollziehbar dokumentiert sind. Besonders in komplexen Microservice-Architekturen, in denen viele Services miteinander interagieren, bietet Contract Testing eine skalierbare Möglichkeit, die Qualität und Konsistenz der Schnittstellen zu gewährleisten [41].

Trotz dieser Vorteile bringt Contract Testing auch einige Herausforderungen und potenzielle Nachteile mit sich. So wie die Kollaboration zwischen Teams immer Abstimmungen benötigt, erfordert auch die erfolgreiche Umsetzung von Contract Testing eine enge Koordination zwischen den beteiligten Teams, da Änderungen an den Contracts sowohl auf Consumer- als auch auf der Provider-Seite abgestimmt und kommuniziert werden müssen. In großen Organisationen oder bei der Zusammenarbeit mit externen Partnern kann dies anfänglich zu einem erhöhten Abstimmungsaufwand führen [18]. Zudem ist die kontinuierliche Pflege und Versionierung der Contracts notwendig, um sicherzustellen, dass die Tests stets den aktuellen Stand der Schnittstellen widerspiegeln. Ohne klare Prozesse und geeignete Werkzeuge zur Contract-Verwaltung besteht die Gefahr, dass veraltete oder widersprüchliche Contracts die Integrität des Systems beeinträchtigen [11].

Ein weiterer Nachteil ist die begrenzte Testtiefe von Contract Tests. Sie überprüfen in erster Linie die Einhaltung der syntaktischen und strukturellen Vorgaben der Schnittstellen, bieten jedoch keine Garantie für die korrekte semantische Umsetzung der Funktionalität [28]. So kann es beispielsweise vorkommen, dass ein Service zwar formal korrekte Daten liefert, diese aber inhaltlich fehlerhaft sind. Solche Probleme werden durch Contract Testing allein nicht erkannt und erfordern ergänzende Testmethoden, wie etwa Unit- oder Integrationstests, wie in Abschnitt 2.2 beschrieben, die die Geschäftslogik und die funktionalen Anforderungen abdecken. Contract Testing sollte daher nicht als Ersatz für andere Testmethoden betrachtet werden, sondern vielmehr als ergänzende Maßnahme, die in eine umfassende Teststrategie integriert werden sollte [8].

Nicht zuletzt ist der initiale Implementierungsaufwand für Contract Testing nicht zu unterschätzen. Die Einführung entsprechender Tools, die Schulung der Teams und die Anpassung bestehender Entwicklungs- und Testprozesse können insbesondere in großen, gewachsenen Systemlandschaften herausfordernd sein [11]. Auch die Integration von Contract Testing in bestehende CI/CD-Pipelines und die regelmäßige Aktualisierung der Tests erfordern zusätzliche Ressourcen und Disziplin im Entwicklungsalltag.

Zusammenfassend lässt sich festhalten, dass Contract Testing einen wertvollen Beitrag zur Qualitätssicherung von API-Kommunikation leisten kann. Die Methode ermöglicht eine frühzeitige Fehlererkennung, fördert die Dokumentation und beschleunigt Entwicklungsprozesse. Gleichzeitig setzt sie jedoch eine sorgfältige Integration in bestehende Abläufe sowie eine kontinuierliche Zusammenarbeit und Abstimmung zwischen den beteiligten Teams voraus, um die damit verbundenen Herausforderungen erfolgreich zu bewältigen. Vor allem in Microservice-Architekturen, in denen die Interaktion zwischen vielen unabhängigen Services von unterschiedlichen Teams im Vordergrund steht, erweist sich Contract Testing als effektives Mittel zur Sicherstellung der Schnittstellenkompatibilität und zur Vermeidung von Integrationsproblemen [41]. Contract Testing sollte daher stets als ergänzende Maßnahme zu anderen Testmethoden verstanden und im Rahmen einer ganzheitlichen Teststrategie eingesetzt werden, um sowohl die strukturelle Integrität als auch die funktionale Korrektheit von Softwaresystemen sicherzustellen.

Kapitel 3

Contract Testing Frameworks

Nachdem die grundlegenden Konzepte von Contract Testing dargestellt wurden, soll im Rahmen dieser Arbeit untersucht werden, wie diese Testmethode in der Praxis konkret umgesetzt werden kann. Insbesondere die Auswahl geeigneter Werkzeuge spielt eine zentrale Rolle für die erfolgreiche Implementierung und Integration von Contract Testing in bestehende Entwicklungsprozesse. In diesem Kapitel werden daher verschiedene Contract Testing Tools systematisch analysiert und hinsichtlich ihrer Funktionalitäten, Einsatzmöglichkeiten und spezifischen Stärken sowie Schwächen bewertet. Ziel ist es, einen fundierten Überblick über die verfügbaren Lösungen zu geben und ein passendes Tool für die prototypische Implementierung in Kapitel 4 auszuwählen.

3.1 Übersicht verfügbarer Frameworks

Die Identifikation geeigneter Frameworks für die prototypische Umsetzung der Contract Tests erfolgte auf Basis definierter Anforderungen, die in Abstimmung mit dem betreuenden Unternehmen *msg systems ag* erarbeitet wurden. Zwei Bedingungen wurden dabei als zwingend erforderlich festgelegt: Zum einen sollte das jeweilige Tool unter einer Open-Source-Lizenz stehen, um eine transparente Einsicht in den Quellcode sowie die Unabhängigkeit von einem Anbieter zu gewährleisten. Zum anderen wurde gefordert, dass das Framework explizit die CDCT-Prinzipien aus Abschnitt 2.4.2 unterstützt, da dieses Modell im vorliegenden Anwendungskontext einer Microservice-Architektur eingesetzt werden soll.

Im Verlauf dieser Recherche konnten fünf der untersuchten Frameworks identifiziert werden, die sowohl den Open-Source-Ansatz verfolgen als auch eine explizite Unterstützung von CDCT bieten. Diese werden im Folgenden vorgestellt. Die in die nähere Analyse einbezogenen Tools sind **Pact**, **Spring Cloud Contract**, **Specmatic**, **Karate** und **ContractCase**. Diese Tools bilden die Grundlage für die nachfolgende vergleichende Analyse und Auswahl in Abschnitt 3.3.

Pact

Pact¹ ist eines der verbreitetsten Frameworks im Bereich des Contract Testing. Das Tool wurde erstmals im Jahr 2013 veröffentlicht und ist ein Open-Source-Projekt, das von der australischen Softwarefirma DiUS ins Leben gerufen wurde. Ziel war es, eine leichtgewichtige und praktikable Lösung für API-Testing in einer Microservice-Architektur bereitzustellen [36]. Die Entwicklung wird heute von der gemeinnützigen **Pact Foundation** koordiniert, die eine wachsende Open-Source-Community unterstützt. Es existieren offizielle Implementierungen für mehrere Programmiersprachen und einen zentralen Broker zur Verwaltung von Contracts [36]. Es gibt auch ein eigenes Hosting-Angebot des **Pact Brokers** namens **Pactflow**, welches allerdings kommerziell lizenziert ist und nicht quelloffen zur Verfügung steht [25] und deshalb nicht in der funktionellen Analyse berücksichtigt wurde.

Spring Cloud Contract

Spring Cloud Contract² ist Teil des umfassenden Spring-Cloud-Projekts und wurde 2016 unter dem Dach von Pivotal (heute VMware Tanzu) veröffentlicht [47]. Es ist speziell auf das Spring-Ökosystem zugeschnitten und integriert sich nahtlos in bestehende Java-Anwendungen. Das Projekt wird kontinuierlich durch die Entwickler bei VMware gepflegt und ist Teil der offiziellen Spring-Infrastruktur. Dank der starken Position von Spring im Java-Enterprise-Umfeld ist **Spring Cloud Contract** weit verbreitet und wird aktiv weiterentwickelt [46].

Specmatic

Specmatic³ wurde 2019 vorgestellt und ist ein CDCT-Tool, welches im Kern open-source ist, aber auch kommerzielle Angebote bietet [9]. Es entstand aus dem Bestreben, API-Spezifikationen wie OpenAPI als zentrale Contract-Quelle zu nutzen, und daraus sowohl Tests als auch virtuelle Services zu generieren. Das Framework unterstützt eine Vielzahl von Protokollen und verfolgt einen No-Code-Ansatz, um die Implementierung von Contract Tests zu erleichtern [9]. **Specmatic** bietet eine quelloffene Basis, die durch einige kommerzielle Features erweitert wird [9]. In der nachfolgenden Analyse wird **Specmatic** als Open-Source-Tool betrachtet, da die Kernfunktionen für Contract Testing ohne kommerzielle Komponenten genutzt werden können.

¹<https://pact.io>

²<https://spring.io/projects/spring-cloud-contract>

³<https://specmatic.io>

Karate

Karate⁴ wurde ab 2017 von einer Einzelperson als Open-Source-Projekt veröffentlicht. Ziel war es, ein umfassendes und einfach zu verwendendes Framework für die API-Test-Automatisierung zu schaffen. **Karate** wurde später unter dem Dach von **Karate Labs** weiterentwickelt, einem dedizierten Unternehmen zur Professionalisierung und Kommerzialisierung des Tools [34]. Trotz kommerzieller Angebote ist der Kern von **Karate** weiterhin quelloffen und wird von einer aktiven Community getragen. In dieser Arbeit werden nur die Open-Source Funktionalitäten von **Karate** als Open-Source-Tool betrachtet. Die Bekanntheit des Tools hat sich in den letzten Jahren durch gute Dokumentation und eine niedrige Einstiegshürde deutlich erhöht [23].

ContractCase

ContractCase⁵ ist ein vergleichsweise neues Projekt, das seit 2022 auf GitHub gepflegt wird. Es wird von einer Einzelperson inspiriert von **Pact** entwickelt und zielt darauf ab, Contract Testing in einer modernen, leichtgewichtigen Weise zu ermöglichen [7]. Obwohl **ContractCase** bislang weniger verbreitet ist als andere Frameworks, zeigt die wachsende Nutzerbasis ein zunehmendes Interesse in der Entwicklergemeinschaft [1].

Für die Untersuchung hinsichtlich ihrer Relevanz für Contract Testing werden im Folgenden die Kriterien beschrieben, welche der vergleichenden Gegenüberstellung der fünf Frameworks in Abschnitt 3.3 als Grundlage dienen.

3.2 Auswahlkriterien

Die Auswahl geeigneter Contract-Testing-Frameworks setzt eine methodisch fundierte Bewertung voraus, die sich an spezifischen Anforderungen und Zielsetzungen des Contract Testing orientiert. Im Rahmen dieser Arbeit wurden die Kriterien in enger Zusammenarbeit mit dem betreuenden Unternehmen *msg systems ag* entwickelt, um sicherzustellen, dass sowohl technische als auch betriebliche Aspekte angemessen berücksichtigt werden. Die Kriterien sollen dabei nicht nur die Funktionalitäten eines Tools erfassen, sondern auch Aspekte wie Wartbarkeit, Zukunftssicherheit und organisatorische Integration reflektieren. Außerdem wurden die einzelnen Kriterien zusätzlich gewichtet, um deren relative Bedeutung für die Toolbewertung im konkreten Anwendungskontext zu verdeutlichen. Die einzelnen Gewichtungsfaktoren sind in Abschnitt 3.3 in Tabelle 3.1 zu finden. Die nachfolgenden

⁴<https://www.karatelabs.io>

⁵<https://case.contract-testing.io>

Ausführungen erläutern die einzelnen Kriterien im Detail und legen deren Relevanz für die Toolbewertung dar.

3.2.1 Technisch-funktionale Aspekte

Hierunter fallen alle funktionalen Merkmale, die das Tool in Bezug auf die Durchführung von Contract Tests bietet. Bewertet wurden unter anderem die Unterstützung synchroner und asynchroner Kommunikation sowie die Möglichkeit zur Validierung verschiedener Schnittstellentypen (Representational State Transfer (REST), GraphQL, WebSocket, Kafka, Remote Procedure Call (RPC), usw.). Auch die unterstützten Varianten von Contract Testing, wie Abschnitt 2.4.2 erklärt, wurden berücksichtigt. Zudem wurde die Möglichkeit, verschiedene Anforderungen in den Contracts zu definieren, wie etwa Validierung von Request/Response-Strukturen, Headern, Statuscodes, usw., bei der Bewertung einbezogen.

3.2.2 Sprach- und Technologieunterstützung

Ein weiteres zentrales Kriterium ist die Unterstützung von unterschiedlichen Programmiersprachen und Technologien. Frameworks, die Multi-Language-Support oder spezifische Integrationen für Java, JavaScript, Python, etc. bieten, sind flexibler einsetzbar und ermöglichen eine schnellere und nahtlosere Adaption in die unterschiedlichen Microservices.

3.2.3 Initialer Aufwand

Der Aufwand zur erstmaligen Implementierung eines Contract-Testing-Tools ist ein wesentlicher Faktor bei der Bewertung. Er umfasst sowohl den technischen Integrationsaufwand in bestehende Anwendungen und CI/CD-Umgebungen als auch die Komplexität der Erstellung der Contract Tests an sich. Dazu zählen z.B. die Einarbeitung in die Funktionsweise des Tools, das Installieren und Konfigurieren der Plugins, das Definieren von Mock-Servern sowie die Erstellung von Überprüfungskriterien. Tools mit einfacher Einstiegshilfe, klarer Struktur und guter Dokumentation zur Erstkonfiguration bieten in diesem Bereich Vorteile.

3.2.4 Kontinuierlicher Aufwand

Neben dem initialen Aufwand ist auch der laufende Wartungsaufwand zu berücksichtigen. Dieser umfasst etwa die Aktualisierung von Contracts bei Änderungen der Schnittstellen, das Nachziehen von Tests und die Pflege von Testdaten. Ein gutes Tool sollte Mechanismen zur Reduktion dieses Aufwands bieten, beispielsweise durch automatische Generierung

3.2 Auswahlkriterien

von Tests aus bestehenden Contracts oder durch unterstützende Integration in die Entwicklungsumgebung. Ein geringer kontinuierlicher Wartungsaufwand verbessert die langfristige Wirtschaftlichkeit und Nachhaltigkeit des Tools im Betrieb.

3.2.5 Lizenz und Kosten

Ein zentraler Aspekt war die Lizenzierung der Tools. Da im betrachteten Anwendungsfall ausschließlich Open-Source-Lösungen infrage kamen, wurde überprüft unter welchen Open-Source-Lizenzen die Frameworks stehen, um so Rechte und Pflichten der Nutzung zu klären. Zusätzlich wurden mögliche Kosten durch optionale kommerzielle Komponenten, Support-Angebote oder Hosting-Dienste mit in die Bewertung einbezogen.

3.2.6 Aktualität und Weiterentwicklung

Ein Indikator für die Zukunftssicherheit eines Frameworks ist die Aktivität in der Weiterentwicklung. Bewertet wurden die Häufigkeit von Releases, die Anzahl aktiver Kontributoren, die Pflege von Issue-Trackern und Pull Requests sowie die Reaktionszeit auf Sicherheitslücken oder Fehlerberichte. Auch die Existenz einer aktiven Community oder einer verantwortlichen Organisation sind relevant, da diese langfristig Wartung und Support wahrscheinlicher machen.

3.2.7 Dokumentation

Die Qualität der verfügbaren Dokumentation hat direkten Einfluss auf die Nutzbarkeit und den Einarbeitungsaufwand eines Frameworks. Bewertet wurden Umfang, Struktur und Aktualität der offiziellen Dokumentation sowie zusätzliche Ressourcen wie Tutorials, Blogs oder Community-Foren. Eine gut strukturierte und vollständige Dokumentation erleichtert sowohl den Einstieg als auch die Lösung von Problemen im laufenden Betrieb.

3.2.8 Contract-Verwaltung

Ein zentrales Element des Contract Testings ist das Management von Contracts zwischen Consumer und Provider. Die Tools wurden dahingehend bewertet, ob sie Funktionen zur zentralen Speicherung, Versionierung, Prüfung und Verteilung von Contracts anbieten. Berücksichtigt wurde auch, ob es Unterstützung für die Verwaltung von Contracts gibt. Einige Frameworks bieten integrierte Broker-Funktionalitäten, die eine zentrale Anlaufstelle für alle Contracts darstellen und den Austausch zwischen Consumer und Provider erleichtern. Bei

diesem Kriterium wurde vor allem die Schnittstelle zwischen den Tests und dem Broker betrachtet. Positiv bewertet wurden Frameworks, die eine einfache Integration mit dem Broker oder umfangreiche Konfigurationsmöglichkeiten bieten, um die Verwaltung von Contracts zu optimieren.

3.2.9 Contract-Versionierung

Da sich APIs im Lebenszyklus eines Softwareprodukts weiterentwickeln, ist eine zuverlässige Versionsverwaltung von Contracts unabdingbar. Bewertet wurde, ob ein Tool den Umgang mit mehreren Versionen eines Contracts gleichzeitig erlaubt und wie diese voneinander abgegrenzt und verwaltet werden können. Idealerweise wird durch Versionierung auch die Rückwärtskompatibilität von Contracts gewährleistet und dokumentiert, um verschiedene Arbeitsstände von APIs testen zu können.

3.2.10 Visualisierung

Einige Tools bieten Funktionen zur Visualisierung von API-Strukturen, Abhängigkeiten und Contracts. Solche Visualisierungen sind insbesondere im Zusammenspiel mit umfangreichen Microservice-Architekturen hilfreich, um die Übersicht zu behalten und Fehlerursachen schneller zu identifizieren. Gute Visualisierungen von Contracts können auch teilweise als API-Dokumentationen verwendet werden und so den Aufwand manueller Erstellung einsparen.

3.2.11 Verbindlichkeit und Fehlererkennungsrate

Die Wirksamkeit eines Contract-Testing-Frameworks steht und fällt mit der Fähigkeit, Verstöße gegen definierte Contracts zuverlässig zu erkennen. Bewertet wurde, wie präzise und vollständig die Tools Inkonsistenzen identifizieren und wie aussagekräftig die resultierenden Fehlermeldungen sind. Ein hoher Grad an Verbindlichkeit stärkt das Vertrauen in die Tests und reduziert das Risiko von Integrationsfehlern.

3.2.12 Automatisierbarkeit

Ein weiteres zentrales Ziel des Einsatzes eines Frameworks ist die möglichst nahtlose Integration in automatisierte Entwicklungs- und Deployment-Prozesse. Daher wurde untersucht, inwieweit die einzelnen Frameworks eine Automatisierung der Vertragserstellung und -überprüfung im Rahmen von CI/CD-Pipelines ermöglichen. Besonderes Augenmerk galt hierbei der Unterstützung gängiger Build-Tools (z. B. `Maven`, `Gradle`, `Node Package`

3.3 Vergleich der Frameworks

Manager (npm)), CI/CD-Server (z. B. Jenkins, Drone CI, GitHub Actions) sowie der Möglichkeit, Validierungsprozesse durch Webhooks oder Command Line Interface (CLI)-Befehle zu automatisieren.

3.2.13 Skalierbarkeit

Zuletzt wurde auch die Skalierbarkeit der Tools bewertet. Dies betrifft die technische Fähigkeit, mit wachsender Anzahl an Services und Contracts umzugehen, aber auch organisatorische Aspekte wie die Nutzbarkeit durch mehrere Teams oder Integration in größere Systemlandschaften. Tools, die eine flexible und performante Nutzung auch in großen Umgebungen ermöglichen, wurden in dieser Kategorie positiv bewertet.

Die Auswahl der hier genannten Kriterien bildet die Grundlage für die vergleichende Bewertung der in Abschnitt 3.1 vorgestellten Frameworks im nachfolgenden Abschnitt.

3.3 Vergleich der Frameworks

Basierend auf den in Abschnitt 3.2 definierten Kriterien wurde eine vergleichende Analyse der in Abschnitt 3.1 vorgestellten Contract-Testing-Tools durchgeführt. Die Bewertung erfolgt für jedes Tool entlang der einzelnen Kriterien, um eine fundierte Entscheidungsgrundlage für die spätere Implementierung bereitzustellen. Für jedes Kriterium wurde eine Bewertungspunktzahl von 1 bis 5 vergeben, wobei 1 für eine unzureichende Erfüllung des Kriteriums und 5 für eine hervorragende Erfüllung steht. Durch Multiplizieren der Punktzahl des Kriteriums mit dem Gewichtungsfaktor ergibt sich eine gewichtete Bewertung, die als Summe die Relevanz des Frameworks für Contract Testing im Anwendungskontext dieser Arbeit widerspiegelt. Die Gewichtungsfaktoren wurden durch firmeninterne Abstimmungen bestimmt und spiegeln somit die Wichtigkeit der einzelnen Kriterien für das Unternehmen wider, in dem die Contract Tests eingesetzt werden sollen.

Pact

Technisch-funktionale Aspekte: Pact unterstützt synchrone HTTP-Kommunikation über REST, Simple Object Access Protocol (SOAP), JSON-RPC, GraphQL sowie synchrone als auch asynchrone Schnittstellen (z. B. Kafka, WebSocket, Message Queuing Telemetry Transport (MQTT) usw.) [36]. Es bietet umfassende Möglichkeiten zur Definition von Contracts, einschließlich Validierung von Request/Response-Strukturen, Headern und Statuscodes sowie verschiedene Typ- und Format- und Wertüberprüfungen [36].

Sprach- und Technologieunterstützung: Pact bietet breite Sprachunterstützung mit vielen Plugins für insgesamt über 12 Programmiersprachen-Umgebungen u. a. Java, JavaScript/TypeScript, Python, Ruby, Go, Rust, C++ [35].

Initialer Aufwand: Die erstmalige Einrichtung ist gut dokumentiert, jedoch erfordert die Vielfalt an Implementierungen eine gewisse Einarbeitung, da sich die Integration in verschiedene Anwendungskontexte sich teilweise stark unterscheidet [36].

Kontinuierlicher Aufwand: Änderungen an Contracts müssen händisch nachgezogen werden, was bei vielen Services aufwendig sein kann. Welche Contracts und welche Validierungen konkret angepasst werden müssen, wird allerdings transparent dargestellt. Die Provider-Tests werden hingegen automatisch aus den Consumer-Contracts generiert, was den Aufwand reduziert [36].

Lizenz und Kosten: Pact steht unter der Massachusetts Institute of Technology (MIT)-Lizenz [35], eine Open-Source-Lizenz, die freie Nutzung, Änderung und Weitergabe mit Lizenzhinweis erlaubt [26]. Kommerzielle Zusatzangebote wie PactFlow bieten optionale Erweiterungen und Cloud-Dienste [39].

Aktualität und Weiterentwicklung: Pact weist eine hohe Entwicklungsaktivität auf, mit regelmäßigen Updates in den verschiedenen Sprachimplementierungen [35]. Außerdem wird die Entwicklung von der Pact Foundation koordiniert und durch das Unternehmen PactFlow unterstützt [39].

Dokumentation: Die Dokumentation ist umfangreich und deckt wichtige Anwendungsfälle ab und ist trotz vieler unterschiedlicher Repositories auf einer zentralen Webseite bereitgestellt [36].

Contract-Verwaltung: Mit Pact Broker ist eine zentrale Verwaltung und Veröffentlichung von Contracts möglich. Dieser ist mithilfe der unterschiedlichen Plugins gut in die verschiedenen Programmiersprachen-Umgebungen integrierbar [36].

Contract-Versionierung: Pact Broker unterstützt Versionierung und die Historisierung von Contracts. Zudem können verschiedene und auch neue Contract-Versionen durch den Broker (vor-)verifiziert werden, um so die Versionskompatibilität zu erhöhen [36].

Visualisierung: Pact Broker bietet übersichtliche Web-Oberflächen für Vertragszustände und Verifizierungen. Darüber hinaus lässt sich eine Übersicht über die Abhängigkeiten zwischen Consumer und Provider generieren [36].

Verbindlichkeit und Fehlererkennungsrate: Pact liefert detaillierte Fehlermeldungen und bietet eine zuverlässige Erkennung von Vertragsbrüchen. Durch das separate `can-i-deploy`-Feature kann die Verbindlichkeit von Consumer- und Provider-Contracts überprüft werden, bevor Deployments durchgeführt werden [36].

3.3 Vergleich der Frameworks

Automatisierbarkeit: Pact unterstützt CI/CD-Integrationen durch gute Kompatibilität mit typischen Build-Tools und einem CLI-Tool [36].

Skalierbarkeit: Mit der Cloudfähigkeit und strukturierter Contract-Verwaltung des **Pact Brokers** lassen sich auch größere Projekte gut abbilden und effizient testen [36].

Spring Cloud Contract

Technisch-funktionale Aspekte: Spring Cloud Contract fokussiert sich auf synchrone HTTP-Kommunikation über REST und bietet auch (teilweise nur experimentelle) Unterstützung für weitere Protokolle wie GraphQL, gRPC sowie Messaging-Protokolle [48].

Sprach- und Technologieunterstützung: Der Fokus liegt hier stark auf Java/Spring-Anwendungen. Daher gibt es Unterstützung für Java, Groovy und Kotlin, aber keine explizite Unterstützung für andere Sprachen [48].

Initialer Aufwand: Die Integration in bestehende Spring-Projekte ist standardisiert, da es direkt in das Spring-Ökosystem eingebunden ist [46].

Kontinuierlicher Aufwand: Die gute Integration mit Spring erleichtert die Wartung, da die Tests auf Provider-Seite vollständig automatisiert aus dem Contract generiert werden [48].

Lizenz und Kosten: Spring Cloud Contract ist unter der Apache-2.0-Lizenz veröffentlicht [49], die eine freie und auch kommerzielle Nutzung und Modifikation der Software erlaubt [3].

Aktualität und Weiterentwicklung: Als Teil des Spring-Cloud-Projekts wird es aktiv gepflegt [46], befindet sich aber zum Zeitpunkt der Erstellung dieser Arbeit nicht mehr unter starker Weiterentwicklung [49].

Dokumentation: Die offizielle Dokumentation ist gut strukturiert, setzt jedoch teilweise tiefere Spring-Kenntnisse voraus [48].

Contract-Verwaltung: Die Verwaltung von Contracts erfolgt über ein zentrales Source-Control-Repository, in dem die Contracts als Groovy- oder YAML Ain't Markup Language (ehemals Yet Another Markup Language) (YAML)-Dateien abgelegt werden [48].

Contract-Versionierung: Die Versionierung muss mithilfe von Build-Tools oder externen Repositories manuell organisiert werden [48].

Visualisierung: Es steht keine eigene Visualisierungskomponente zur Verfügung [48].

Verbindlichkeit und Fehlererkennungsrate: Spring Cloud Contract zeichnet sich durch eine hohe Verbindlichkeit aus, da API-Implementierungen strikt gegen definierte Contracts getestet werden [48]. Die Fehlererkennungsrate ist hoch, sofern Contracts vollständig gepflegt sind und die tatsächliche Nutzung der APIs abbilden.

Automatisierbarkeit: Die Integration in Spring-Build-Prozesse (Maven oder Gradle) wird unterstützt, was eine einfache CI/CD-Anbindungen ermöglicht [48].

Skalierbarkeit: Durch die enge Integration in das Spring-Ökosystem ist das Tool gut skalierbar, da der Fokus auf Cloud- und Microservice-Umgebungen liegt [48].

Specmatic

Technisch-funktionale Aspekte: Specmatic unterstützt in der Open-Source-Variante lediglich die Kommunikation über REST und SOAP [9]. Aufgrund der OpenAPI-Basis können Contracts verschiedene Validierungsregeln definieren, einschließlich der Typ- und Format- und Wertüberprüfungen [33].

Sprach- und Technologieunterstützung: Specmatic ist sprachunabhängig und kann in verschiedenen Umgebungen eingesetzt werden, da Tests mithilfe eines alleinstehenden CLI-Tools durchgeführt werden [45].

Initialer Aufwand: Die manuelle Erstellung von OpenAPI-Spezifikationen ist notwendig. Die Integration in bestehende Projekte erfordert eine gewisse Einarbeitung, da es keine spezifischen Plugins für gängige Build-Tools gibt [45] und muss daher manuell durchgeführt werden.

Kontinuierlicher Aufwand: Die OpenAPI-Spezifikationen müssen bei Änderungen der Schnittstellen manuell aktualisiert werden, da diese nicht mit dem Anwendungscode zusammenhängen, was den kontinuierlichen Aufwand in der Praxis erhöht [45].

Lizenz und Kosten: Specmatic steht unter MIT-Lizenz [54], die eine freie Nutzung und Modifikation erlaubt [26]. Specmatic bietet jedoch auch kommerzielle Erweiterungen an, welche z. B. weitere Protokolle unterstützen [9].

Aktualität und Weiterentwicklung: Das Tool ist aktiv in der Entwicklung, wird jedoch zum Zeitpunkt der Erstellung dieser Arbeit mit einem kleinen Team entwickelt und hat eine kleine Community [54]. Viele Weiterentwicklungen sind ausschließlich auf die kommerzielle Version ausgerichtet [9].

Dokumentation: Die Dokumentation ist gut strukturiert und bietet viele Beispiele, ist jedoch nicht so umfangreich wie bei den anderen Frameworks [45].

3.3 Vergleich der Frameworks

Contract-Verwaltung: *Specmatic* bietet keine zentrale Contract-Verwaltung, sondern nutzt OpenAPI-Spezifikationen, die in Repositories manuell verwaltet werden [9].

Contract-Versionierung: Die Versionierung erfolgt manuell über Git oder andere Versionskontrollsysteme, allerdings ohne eine spezielle Unterstützung durch das Tool [45].

Visualisierung: Die standardisierten OpenAPI-Spezifikationen können als API-Dokumentationen visualisiert werden [33]. Die Testergebnisse können in Form von JSON- oder eXtensible Markup Language (XML)-Berichten ausgegeben werden, jedoch ist mit dem Tool keine grafische Visualisierung dieser möglich [45].

Verbindlichkeit und Fehlererkennungsrate: *Specmatic* bietet eine hohe Verbindlichkeit, da es auf standardisierten OpenAPI-Spezifikationen basiert und diese strikt gegen die Implementierung prüft [45].

Automatisierbarkeit: Die Automatisierung erfolgt über ein CLI-Tool, das in CI/CD-Pipelines integriert werden kann. Dies erfordert allerdings manuelle Integration in bestehende Abläufe, da es keine spezifischen Plugins für gängige Build-Tools gibt [45].

Skalierbarkeit: *Specmatic* ist aufgrund seiner sprachunabhängigen Natur und der Nutzung von OpenAPI-Spezifikationen gut skalierbar. Allerdings kann die Contract-Verwaltung bei größeren Projekten aufwendig und unübersichtlich werden, da sie nicht zentralisiert ist und manuelle Pflege erfordert [45].

Karate

Technisch-funktionale Aspekte: *Karate* unterstützt synchrone HTTP-Kommunikation über REST, SOAP und GraphQL [32]. Es bietet umfassende Möglichkeiten zur Definition von Contracts, einschließlich der Validierung von Request/Response-Strukturen, Headern, Statuscodes und benutzerdefinierten Überprüfungen mithilfe der Domain-Specific Language (DSL) von *Karate* [23].

Sprach- und Technologieunterstützung: *Karate* ist in Java implementiert, bietet jedoch eine eigene DSL, die es ermöglicht, Tests in einer leicht verständlichen und programmiersprachenunabhängigen Syntax zu schreiben. Es bietet eine gute Integration in Java-Umgebungen, ist aber durch ein alleinstehendes CLI-Tool auch in anderen Anwendungsumgebungen einsetzbar [23].

Initialer Aufwand: Die Einrichtung ist einfach, da *Karate* als Maven- oder Gradle-Plugin integriert werden kann. Die Tests können in einer leicht verständlichen DSL geschrieben werden [32], was aber eine gewisse Einarbeitungszeit erfordert.

Kontinuierlicher Aufwand: Der Wartungsaufwand der Tests ist durch die klare Struktur und die Möglichkeit, Tests in einer DSL zu schreiben, relativ gering [23].

Lizenz und Kosten: Karate ist quelloffen und steht unter der MIT-Lizenz [23], die eine freie Nutzung und Modifikation erlaubt [26].

Aktualität und Weiterentwicklung: Karate ist aktiv in der Entwicklung, mit seltenen, aber regelmäßigen Updates und einer großen Community [23]. Die Entwicklung wird von Karate Labs unterstützt, die auch kommerzielle Erweiterungen anbieten [34].

Dokumentation: Die Dokumentation ist umfangreich und gut strukturiert. Sie bietet Beispiele, Tutorials und weitere Ressourcen für die Nutzung des Tools und der DSL [23].

Contract-Verwaltung: Karate bietet keine zentrale Contract-Verwaltung, sondern nutzt eine einfache Dateistruktur zur Ablage der Contracts [23]. Diese können in einem Repository verwaltet werden, jedoch ohne spezielle Unterstützung durch das Tool selbst.

Contract-Versionierung: Zwar ist eine Versionierung über gängige Versionskontrollsysteme wie Git möglich, allerdings bietet das Tool hierfür keine spezielle Funktionalität [23].

Visualisierung: Karate bietet eine einfache Visualisierung der Testergebnisse in Form von HTML-Berichten, die jedoch keine umfassende API-Dokumentation ersetzen können [23]. Darüber hinaus gibt es keine speziellen Visualisierungsfunktionen für Contracts und API-Abhängigkeiten o. ä.

Verbindlichkeit und Fehlererkennungsrate: Karate bietet eine hohe Verbindlichkeit, da es auf einer klaren DSL basiert und die Tests direkt gegen die Implementierung prüft [23]. Allerdings ist die Fehlererkennungsrate abhängig von der Qualität der Tests und der definierten Contracts.

Automatisierbarkeit: Karate unterstützt die Automatisierung durch Integration in CI/CD-Pipelines über Maven oder Gradle. Die Tests können über ein CLI-Tool ausgeführt werden, was auch eine manuelle Integration in bestehende Abläufe ermöglicht [23].

Skalierbarkeit: Die Tests sind bei Karate aufgrund seiner klaren Struktur und der Möglichkeit, Tests in einer DSL zu schreiben, gut skalierbar [23]. Bei umfangreicheren Projekten gestaltet sich die Contract-Verwaltung dann aufwendig, da es an einer zentralen Verwaltung der Contracts fehlt.

ContractCase

Technisch-funktionale Aspekte: ContractCase ist begrenzt auf das Testen von REST-APIs und bietet grundlegende Validierungsfunktionen für Request/Response-Strukturen, Header und Statuscodes [10].

Sprach- und Technologieunterstützung: ContractCase bietet Unterstützung für Java und JavaScript/TypeScript. Eine Integration in andere Programmiersprachen wie Python,

3.3 Vergleich der Frameworks

C# und Go ist geplant, ist aber zum Zeitpunkt der Erstellung dieser Arbeit noch nicht verfügbar [7].

Initialer Aufwand: Die Einbindung in bestehende Projekte ist gut beschrieben, da es für beide unterstützten Sprachen spezifische Plugins gibt. Die Erstellung der Contracts erfolgt mittels Code, was die Einarbeitung erleichtert, da es anderen Testarten ähnelt [10].

Kontinuierlicher Aufwand: Der Wartungsaufwand der Contracts ist durch die codebasierte Definition relativ gering, da Änderungen direkt im Code nachgezogen werden können [10].

Lizenz und Kosten: `ContractCase` ist unter der Berkeley Software Distribution (BSD)-3-Clause-Lizenz veröffentlicht [7], die die freie Nutzung, Änderung und Weiterverbreitung der Software erlaubt, solange der ursprüngliche Copyright-Hinweis erhalten bleibt [6].

Aktualität und Weiterentwicklung: `ContractCase` ist ein relativ neues Projekt und befindet sich noch im Aufbau. Die Entwicklung erfolgt aktiv mit häufigen Änderungen, jedoch nur durch eine Einzelperson [7].

Dokumentation: Die Dokumentation bietet umfangreiche Informationen zur Nutzung des Tools [10]. Allerdings gibt es aufgrund der geringen Verbreitung des Projekts wenige Beispiele und Tutorials.

Contract-Verwaltung: `ContractCase` ist kompatibel mit dem `Pact Broker`, was eine zentrale Verwaltung und Veröffentlichung von Contracts ermöglicht. Die Unterstützung der Broker-Funktionalitäten ist aber eingeschränkt, da sich die Struktur der Contracts zwischen `ContractCase` und `Pact` unterscheidet [7].

Contract-Versionierung: Die Versionierung der Contracts erfolgt über den `Pact Broker`, der eine zentrale Anlaufstelle für alle Contracts darstellt [7]. Auch hier ist die Unterstützung jedoch eingeschränkt. Eine Vorverifizierung von neuen Contract-Versionen wie bei `Pact` ist z. B. nicht möglich.

Visualisierung: Die guten Visualisierungsfunktionen des `Pact Brokers` können genutzt werden, um die Contracts und deren Abhängigkeiten, aber nicht deren Inhalt, darzustellen [7].

Verbindlichkeit und Fehlererkennungsrate: `ContractCase` bietet eine hohe Verbindlichkeit, da es auf den `Pact`-Standards basiert und die Tests direkt gegen die Implementierung prüft [10].

Automatisierbarkeit: `ContractCase` unterstützt die Automatisierung durch Integration in CI/CD-Pipelines über `Maven` oder `Gradle` und `npm` [10].

Skalierbarkeit: Die Skalierbarkeit ist aufgrund der Unterstützung für den **Pact Broker** gegeben, der eine zentrale Verwaltung und Veröffentlichung von Contracts ermöglicht [7].

Die nachfolgende Tabelle 3.1 fasst die Bewertung der einzelnen Frameworks anhand der definierten, gewichteten Kriterien zusammen. Die Namen der Tools sind aus Platzgründen abgekürzt: **Pact (P)**, **Spring Cloud Contract (SCC)**, **Specmatic (S)**, **Karate (K)** und **ContractCase (CC)**.

Kriterien	Gewichtung	P	SCC	S	K	CC
Technisch-funktionale Aspekte	2	5	4	3	4	4
Sprach- und Technologieunterstützung	3	5	3	4	4	3
Initialer Aufwand	2	3	4	3	4	3
Kontinuierlicher Aufwand	2	4	4	3	4	4
Lizenz und Kosten	1	4	5	4	5	5
Aktualität und Weiterentwicklung	3	5	3	4	4	4
Dokumentation	3	5	4	3	5	4
Contract-Verwaltung	3	5	3	3	3	3
Contract-Versionierung	2	5	3	3	3	4
Visualisierung	1	4	1	3	2	1
Verbindlichkeit und Fehlererkennungsrate	2	5	4	4	4	4
Automatisierbarkeit	3	5	5	3	5	4
Skalierbarkeit	1	5	4	3	3	4
Summe		60	47	43	50	48
Gewichtete Summe		132	102	93	111	105

Tabelle 3.1: Vergleich der Contract-Testing-Frameworks

Diese vergleichende Bewertung bildet die Grundlage für die Auswahl des am besten geeigneten Tools für die prototypische Implementierung von Contract Testing im Rahmen dieser Arbeit, welche in Kapitel 4 erläutert wird. Die gewichtete Summe der Bewertungen zeigt, dass **Pact** unter den gegebenen Voraussetzungen dieser Arbeit das mit Abstand am besten geeignete Framework für Contract Testing ist. An zweiter Stelle steht **Karate** und dahinter folgen fast gleich auf **ContractCase** und **Spring Cloud Contract**. **Specmatic** scheint hier am wenigsten geeignet zu sein. Daher wird **Pact** als Framework für die Implementierung von Contract Testing im nachfolgenden Kapitel ausgewählt.

Kapitel 4

Prototypische Implementierung

Die prototypische Integration von Contract Testing in die Microservice-Anwendung *msg.Sensorik* erfolgt in mehreren Schritten, die in diesem Kapitel detailliert beschrieben werden. Bevor die konkrete Implementierung des ausgewählten Contract-Testing-Frameworks **Pact** erfolgen kann, ist ein Überblick über die zugrunde liegende Anwendungsumgebung notwendig.

4.1 Anwendungsumgebung

Die Beschreibung der Umgebung umfasst sowohl die technische Infrastruktur als auch die verwendeten Werkzeuge, Frameworks und Systemkomponenten, in welche die Contract Tests eingebettet werden sollen. Hier werden die Rahmenbedingungen dargestellt, unter denen die Integration und die Ausführung der Tests erfolgen wird. Dazu zählen insbesondere die eingesetzten Programmiersprachen, Build- und Test-Pipelines, relevante Dienste sowie die Architektur der Kommunikationsbeziehungen zwischen Consumer und Provider. Ein Verständnis für die Anwendungsumgebung ist essenziell, um die getroffenen Implementierungsentscheidungen nachvollziehbar zu machen und die spätere Übertragbarkeit der Ergebnisse auf andere Projekte realistisch einschätzen zu können.

Die Anwendung mit dem Namen *msg.Sensorik* ist ein internes Projekt der msg systems ag, das als Grundlage für die Implementierung der Contract Tests dient. Ziel der Anwendung ist die Erfassung und automatisierte Verarbeitung von Umgebungsdaten in betreuungsintensiven Wohn- oder Pflegeeinrichtungen. Dabei kommen Sensoren zum Einsatz, die beispielsweise geöffnete Fenster oder Türen detektieren und definierte Reaktionen auslösen können. Die Erfassung erfolgt über Sensoren, deren Signale durch ein lokales Gerät mit Smarthome-Schnittstelle gesammelt und an eine cloudbasierte Verwaltungsplattform weitergegeben werden. Diese Plattform übernimmt die zentrale Steuerung und Regelverarbeitung sowie die Bereitstellung grafischer Dashboards und Reporting-Funktionalitäten. Durch Regeldefinitionen kann festgelegt werden, bei welchen Sensorzuständen welche Aktionen ausgelöst werden, etwa das Herunterregeln eines Heizungsthermostats bei geöffnetem Fenster.

Die Anwendung ist in einer Microservice-Architektur aufgebaut, wobei jeder Microservice jeweils eine spezifische Funktionalität bereitstellt und über Schnittstellen miteinander kommuniziert. Insgesamt ist die Anwendung in fünf verschiedene Microservices aufgeteilt:

- **Instanz-Management-Service:** Dieser Service ist für die Erfassung und Verarbeitung von Stammdaten zuständig.
- **Regel-Service:** Hier ist die Logik zur Verarbeitung von Sensordaten und zur Ausführung von Automatisierungsregeln implementiert.
- **Reporting-Service:** Dieser Service speichert den Verlauf von Sensordaten und stellt Berichte für die Visualisierung bereit.
- **Proxy-Service:** Der Proxy Service übernimmt die Kommunikation mit den Sensoren und den Aktoren.
- **Nutzeroberfläche:** Die Nutzeroberfläche ermöglicht die Interaktion mit der Anwendung durch die Benutzer.

Die technische Umsetzung der Anwendung basiert auf einem modernen Technologie-Stack, der für eine modulare und containerisierte Microservice-Architektur optimiert ist.

Die vier Backend-Services (Instanz-Management, Regel-Service, Reporting-Service und Proxy-Service) sind jeweils als eigenständige Spring-Boot-Anwendungen in Java realisiert und werden mit Maven als Build-Tool verwaltet. Diese bieten jeweils eine REST-API an, die von anderen Microservices konsumiert werden können. Jeder dieser Services bringt seine eigene Datenhaltung mit, je nach Anwendungsfall entweder in Form von relationalen Datenbanken, NoSQL-Datenbanken oder Zeitreihendatenbanken. Die Nutzeroberfläche ist mit dem Framework für Webanwendungsentwicklung Angular in TypeScript realisiert und greift über REST-APIs auf die Backend-Services zu.

Die Microservices interagieren untereinander sowohl als Consumer als auch als Provider. Beispielsweise konsumiert der Proxy-Service Daten vom Instanz-Management-Service, fungiert aber auch als Provider für den Regel-Service, während die Nutzeroberfläche lediglich als Consumer auftritt und z. B. die Stammdaten vom Instanz-Management-Service anfragt. Eine Consumer/Provider-Matrix ist in Tabelle 4.1 dargestellt, um die Kommunikationsbeziehungen zwischen den einzelnen Microservices zu veranschaulichen. Diese Kommunikationsbeziehungen bilden die Grundlage für die Definition der Contracts in Abschnitt 4.2.

Die Containerisierung der Service-Bausteine erfolgt mithilfe von **Docker**, wobei die einzelnen Anwendungsteile über **docker-compose** orchestriert werden. Dadurch ist eine konsistente und reproduzierbare Ausführungsumgebung gewährleistet.

4.2 Technische Umsetzung

Service	Provider	Consumer
Instanz-Management	✓	✓
Regel	✓	×
Reporting	✓	×
Proxy	✓	✓
Nutzeroberfläche	×	✓

Tabelle 4.1: Consumer/Provider-Matrix der Microservices

Für die Versionsverwaltung des Codes in den einzelnen Repositories kommt eine zentral vom Unternehmen gehostete **Gitea**-Instanz zum Einsatz. Jeder Microservice befindet sich in einem eigenen Git-Repository, was eine getrennte Entwicklung und Versionierung erlaubt.

Die Continuous Integration erfolgt über **Drone CI**, welches eng mit **Gitea** interagiert. Die Pipeline-Schritte umfassen unter anderem das Ausführen von Unit- und Integrationstests, das Bauen und Versionieren der Anwendung sowie die Überprüfung der Code-Qualität und verschiedener Sicherheitskriterien durch statische Code-Analyse mit dem Tool **SonarQube** [44]. Genau wie **Gitea** sind auch **Drone CI** und **SonarQube** in der Infrastruktur des Unternehmens gehostet.

4.2 Technische Umsetzung

Die Umsetzung der Contract Tests erfolgte iterativ in einer lokalen Entwicklungsumgebung, um eine möglichst isolierte und kontrollierte Integration der einzelnen Frameworks zu gewährleisten. Zunächst musste die lokale Testumgebung der Microservice-Anwendung vorbereitet werden, um die Contract Tests ausführen zu können. Ein zentrales Element der Implementierung war die Einrichtung eines lokalen **Pact Brokers**, der als zentrales Austauschsystem zwischen Consumer- und Provider-Services dienen soll. Der Broker wird in einem **Docker**-Container betrieben, um eine einfache Handhabung und Portabilität für eine eventuelle Migration in die Infrastruktur des Unternehmens sicherzustellen.

4.2.1 Consumer-Contract als Ausgangspunkt

Getreu nach den Prinzipien von CDCT, die in Abschnitt 2.4.2 vorgestellt wurden, erfolgte der Einstieg in die Contract-Test-Implementierung bei einem Consumer-Service. Hier wurde zuerst die Nutzeroberfläche gewählt, welche gemäß Tabelle 4.1 nur als Consumer einiger Backend-APIs fungiert. Zuerst musste eines der Integrations-Plugins für **Pact** ausgewählt werden, welches Consumer-Tests in TypeScript unterstützt. Die Wahl fiel letztendlich auf **pact-js**, welches von der offiziellen **Pact** Dokumentation empfohlen wird [36]. Damit

wurden die Tests im Nutzeroberflächen-Service-Repository in TypeScript formuliert. Die Testklassen wurden in einem separaten Verzeichnis getrennt von der **Angular**-Anwendung innerhalb des Repositories abgelegt, da die **Angular**-Anwendung Schwierigkeiten mit Abhängigkeiten der **pact-js**-Bibliothek hatte. Nähere Informationen zu den Herausforderungen der Einrichtung von Contract Tests in **Angular** sind in Abschnitt 4.3.2 zu finden.

Für die Erstellung der Testfälle wurde sich an den Methoden der Service-Klassen der **Angular**-Anwendung orientiert, da diese die tatsächlichen Interaktionen mit den Backend-Services im Code abbilden. Für jede der genutzten HTTP-Verben (**GET**, **POST**, **PUT**, **DELETE** usw.) der einzelnen API-Pfade wurde ein separater Consumer-Test definiert, der die erwarteten Interaktionen mit dem zugehörigen Provider-Service beschreibt. Als Erstes wurde für jede Interaktion ein Status definiert, der beschreibt, in welchem Status sich der Provider zur Zeit der Provider-Verifizierung befinden muss, um die zugehörige Antwort zurückliefern zu können. Danach wird die Anfrage festgelegt, die der Consumer an den Provider sendet, einschließlich des HTTP-Verbs, des Pfads, der Header und des Inhalts. Die erwartete Antwort des Providers wird ebenfalls spezifiziert, einschließlich des HTTP-Statuscodes, der Header und der erwarteten Antwortdaten. Dabei konnten auch komplexe Datenstrukturen und Validierungsregeln der Antwortinhalte definiert werden, um sich ändernde Werte zu berücksichtigen. Diese Angaben dienen als Grundlage für den Provider-Mock, gegen den die Aufrufe der **Angular**-Anwendung getestet werden.

Außerdem werden Erwartungen definiert, die bei der Ausführung der Consumer-Tests gegen den Provider-Mock erfüllt sein müssen, um die simulierte Interaktion zu überprüfen. So kann sichergestellt werden, dass die **Angular**-Anwendung die erwarteten Antworten korrekt verarbeitet. Nach erfolgreicher Ausführung der Consumer-Tests wird ein **Pact**-Contract generiert, der diese definierten Interaktionen zwischen Consumer und Provider beschreibt und als JSON-Datei speichert. Darin werden der beteiligte Consumer, der Provider, die erwarteten Interaktionen mit den Anfragen und deren entsprechenden Antworten sowie Metadaten festgehalten. Ein Beispiel für einen solchen **Pact**-Contract ist in Listing 4.1 zu finden und wird anschließend näher erläutert.

Listing 4.1: Beispiel für einen **Pact**-Contract

```

1 {
2   "consumer": {
3     "name": "frontend_component"
4   },
5   "interactions": [
6     {
7       "description": "a request for a customer",
8       "pending": false,
9       "providerStates": [

```

4.2 Technische Umsetzung

```
10         {
11             "name": "I have a customer"
12         }
13     ],
14     "request": {
15         "headers": {
16             "Accept": [
17                 "application/json"
18             ]
19         },
20         "method": "GET",
21         "path": "/customers/1"
22     },
23     "response": {
24         "body": {
25             "content": {
26                 "email": "test@msg.group",
27                 "gebaeudeSet": [],
28                 "hausnummer": "14a",
29                 "id": 1,
30                 "name": "msg.systems",
31                 "ort": "Nuernberg",
32                 "plz": 12345,
33                 "strasse": "teststrasse"
34             },
35             "contentType": "application/json",
36             "encoded": false
37         },
38         "headers": {
39             "Content-Type": [
40                 "application/json"
41             ]
42         },
43         "matchingRules": {
44             "body": {
45                 "$": {
46                     "combine": "AND",
47                     "matchers": [
48                         {
49                             "match": "type"
```

```

50         }
51     ]
52 }
53 },
54     "status": {}
55 },
56     "status": 200
57 },
58     "type": "Synchronous/HTTP"
59 }
60 ],
61 "metadata": {
62     "pactRust": {
63         "ffi": "0.4.22",
64         "models": "1.2.3"
65     },
66     "pactSpecification": {
67         "version": "4.0"
68     }
69 },
70 "provider": {
71     "name": "instance_management_component"
72 }
73 }

```

Der vorliegende Pact Contract beschreibt eine vertraglich definierte Interaktion zwischen dem Consumer `frontend_component` und dem Provider `instance_management_component`. Dabei fordert der Consumer per HTTP-GET-Anfrage unter dem Pfad `/customers/1` einen bestimmten Kunden an. Als Voraussetzung gilt der Zustand „I have a customer“, was bedeutet, dass der Provider zu dem Zeitpunkt des API-Aufrufs durch die Contract Tests auf Provider-Seite einen Kunden mit entsprechender ID bereithalten muss. Die erwartete Antwort mit Kundendaten ist hinsichtlich der JSON-Struktur und dem HTTP-Statuscode 200 definiert. Die Antwortstruktur wird durch Typ-basierte Validierungsregeln überprüft, wodurch sichergestellt wird, dass der Provider die erwarteten Datenformate liefert. Außerdem sind noch Metadaten wie z. B. die Version der Pact-Spezifikation enthalten.

Zur Ausführung der Tests im Nutzeroberfläche-Service kam das Test-Framework **Jasmine** zum Einsatz, welches in dieser **Angular**-Anwendung bereits für andere Testarten verwendet wird. Es ermöglicht die Definition von Testklassen und Testfällen, die in der **Node.js**-Umgebung von **Angular** ausgeführt werden können. Nach erfolgreicher Definition und Aus-

führung der Consumer-Tests wurde der erstellte **Pact-Contract** unter Angabe der Service-Version über die **Pact-CLI** an den **Pact Broker** publiziert.

4.2.2 Provider-Verifizierung

Im Anschluss wurde der zugehörige Provider-Test aufseiten des Instanz-Management-Services implementiert. Die auf Spring-Boot basierenden Services verwenden **JUnit5** für die Ausführung von bestehenden Tests, weshalb dieses Framework auch für die Contract Tests verwendet wurde. Die Verifizierung der Consumer-Contracts erfolgte dem **pact-jvm-junit5-Provider-Framework**, welches speziell für die Integration mit **JUnit5** entwickelt wurde. Dazu wurde die entsprechende Abhängigkeit mithilfe von **Maven** in das Projekt eingebunden und Konfigurationen, wie z. B. das Eintragen der Broker-Uniform Resource Locator (URL) durchgeführt. Dieses Framework ermöglicht es, die im **Pact-Contract** definierten Interaktionen gegen die tatsächliche Implementierung des Providers automatisiert zu prüfen. Es musste hierfür eine Testklasse mit einem **TestTemplate** erstellt werden, die die Consumer-Interaktionen aus dem **Pact-Contract** lädt und gegen die Provider-Implementierung validiert. Um die erwarteten Zustände für die Vertragsprüfung (z. B. vorhandene Entitäten in der Datenbank) zu simulieren, wurden sogenannte **State Definitions** im Provider eingerichtet. Dazu musste für jeden Zustand eine Status-Methode definiert werden, die den Provider in den erwarteten Zustand versetzt, bevor der entsprechende Test ausgeführt wird. Nach erfolgreicher Durchführung der Provider-Tests wird der Contract im **Pact Broker** automatisch als verifiziert markiert und Services, die mit diesem Contract kompatibel sind, können deployt werden.

Nach erfolgreicher Implementierung der Contract Tests für die Kommunikation der Nutzeroberfläche mit dem Instanz-Management-Service wurde der Prozess sukzessive für die anderen Microservices wiederholt, wobei nun die Consumer-Seite auch mit dem **pact-jvm-junit5-Framework** implementiert wurde. Bei der in Tabelle 4.1 in Abschnitt 4.1 aufgelisteten Services erfolgte diese Durchführung nach dem gleichen Prozess wie bei der Nutzeroberfläche. Jedoch wurde die Syntax und die spezifischen Implementierungsdetails auf die in diesen Services verwendete Programmiersprache Java angepasst. Für die Veröffentlichung der Contracts wurde dann im Anschluss das CLI-Plugin über **Maven** installiert, um die **Pact-Contracts** an den **Pact Broker** zu übermitteln.

Jede Service-Interaktion durchlief dabei denselben Zyklus: Erst die Definition der Consumer-Seite, danach die Veröffentlichung zum **Pact Broker**, die Implementierung und Ausführung des Provider-Tests und zuletzt die Veröffentlichung des Verifizierungsstatus. Die Contract Tests wurden für alle relevanten Interaktionen zwischen den Microservices implementiert, um die Interoperabilität und Konsistenz der Kommunikation sicherzustellen. Dabei sind insgesamt sechs Contracts zwischen den fünf Microservices entstanden, die die Interaktionen zwischen den Services abbilden. Diese sind mithilfe eines Abhängigkeitsgraphen in

Abbildung 4.1 visualisiert, welcher durch Nutzung des **Pact Brokers** automatisch generiert wurde.

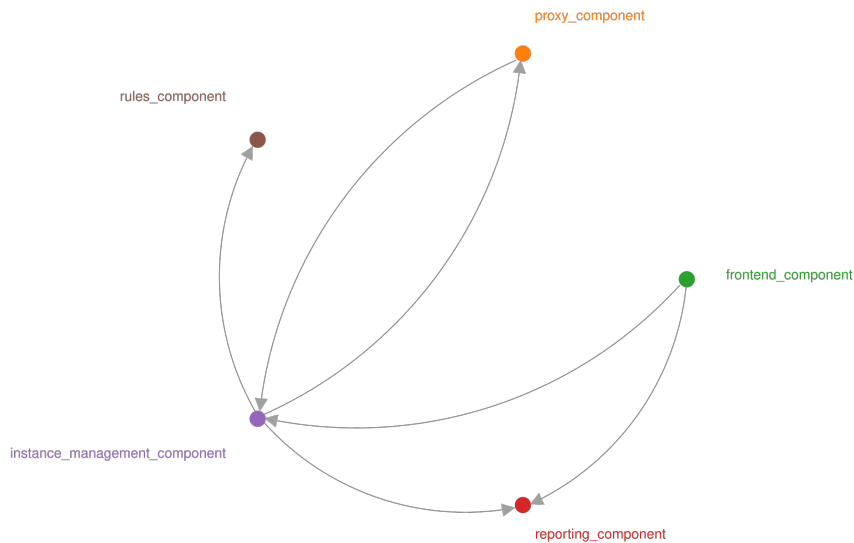


Abbildung 4.1: Abhängigkeitsgraph der Microservices

4.2.3 CI-Integration und Automatisierung

Um die vollständige CI/CD-Pipeline auch abbilden zu können, musste zusätzlich ein lokales **Drone** CI-System aufgesetzt und mit dem **Pact Broker** verbunden werden. Eine Durchführung der Contract Tests in der lokalen Umgebung war notwendig, da die zentrale **Drone** CI-Instanz des Unternehmens aus netzwerktechnischen Gründen nicht ohne Weiteres mit einem lokalen **Pact Broker** verbunden werden kann. Mit dieser lokalen **Drone** CI-Instanz konnte für jeden Microservice eine Pipeline konfiguriert werden, die die Contract Tests automatisch ausführt und deren Ergebnisse im **Pact Broker** speichert. Die Pipeline umfasst je nach Service die folgenden Schritte:

- Ausführen der Consumer-Tests, um den **Pact-Contract** zu generieren.
- Publizieren des **Pact-Contracts** an den **Pact Broker**.
- Ausführen der Provider-Verifizierungstests gegen den **Pact-Contract**.
- Aktualisieren des Verifizierungsstatus im **Pact Broker**.
- **can-i-deploy**-Abfrage an den **Pact Broker**, um zu prüfen, ob der Service mit den aktuellen Contracts kompatibel ist.

4.3 Aufgetretene Herausforderungen

Die Automatisierung dieser Schritte ermöglicht eine nahtlose Integration der Contract Tests in den Entwicklungsprozess und stellt sicher, dass Änderungen an den Services sofort auf ihre Interoperabilität geprüft werden.

4.3 Aufgetretene Herausforderungen

Die Einführung von Contract Tests im Projektumfeld brachte eine Reihe technischer und konzeptioneller Herausforderungen mit sich, wie schon in Abschnitt 4.2 angedeutet, die im Verlauf der Implementierung identifiziert und schrittweise gelöst wurden. Die folgenden Abschnitte skizzieren die zentralen Problemfelder und beschreiben, wie diesen begegnet wurde.

4.3.1 CI/CD-Infrastrukturkomplexität

Eine zentrale Hürde zu Beginn der Umsetzung war die Einrichtung einer funktionsfähigen lokalen Testumgebung, die alle beteiligten Komponenten – insbesondere den **Pact Broker** und das CI/CD-System **Drone CI** und die Versionsverwaltung **Gitea** – umfasst. Die drei Dienste sollten so konfiguriert werden, dass die vollständige Kommunikationskette zwischen Consumer-Tests, **Pact**-Veröffentlichung und Provider-Verifizierung ohne manuelle Zwischenschritte möglich ist.

Da für die prototypische Implementierung keine zentrale **Pact Broker**-Instanz zur Verfügung stand, wurde eine lokale Instanz aufgesetzt. Um die Integration in die **Drone CI**-Pipeline zu ermöglichen, wurde auch die **Drone CI**-Instanz lokal betrieben, da die zentrale **Drone CI**-Instanz des Unternehmens standardmäßig nicht auf den lokalen **Pact Broker** zugreifen kann. Ein Server kann nicht direkt auf einen lokalen Rechner zugreifen, da dies durch grundlegende Prinzipien der Netzwerksicherheit und -architektur verhindert wird. Lokale Rechner befinden sich zum Schutz idealerweise hinter Firewalls und nutzen private IP-Adressen, die durch Network Address Translation vom öffentlichen Internet abgeschildert sind [29].

Die Kommunikation zwischen der zentralen **Gitea**-Instanz und der lokalen **Drone CI**-Instanz ist nötig, um den aktuellen Stand der Repositories zu erhalten und auf Ereignisse reagieren zu können. Die meisten Interaktionen zwischen **Gitea** und **Drone CI** gehen von der CI/CD-Pipeline aus, die auf **Gitea** zugreift, um Repositories zu klonen und so den aktuellen Stand des Codes zu erhalten. Allerdings erfolgt die Benachrichtigung der **Drone CI**-Instanz über neue Commits oder Pull-Requests mittels Webhooks von **Gitea** aus [14], was in diesem Fall nicht möglich war, da die lokale **Drone CI**-Instanz nicht von **Gitea** erreicht werden konnte. So konnten die Pipelines nicht automatisch ausgelöst werden und wurden deshalb manuell gestartet. Dieses Vorgehen bei der prototypischen Implementierung beeinträchtigt aber

nicht die grundsätzliche Funktionsweise der Contract Tests und deren Integration in die CI/CD-Pipeline. Die manuelle Auslösung der Pipelines stellt lediglich eine Einschränkung in der Automatisierung dar und beeinträchtigt die Funktionsweise nicht. Bei der Implementierung in der realen Infrastruktur des Unternehmens wäre die zentrale **Drone** CI-Instanz direkt mit der zentralen **Gitea**-Instanz verbunden, sodass das automatische Auslösen der Pipelines funktioniert. Die **Gitea**-Instanz ebenfalls manuell zu installieren wäre zwar möglich gewesen, war in diesem Fall aber nicht zielführend, da dies eine erhebliche Umstellung der Infrastruktur und Redundanz bedeutet hätte. Aus diesem Grund wurde die Lösung mit dem zentralen **Gitea**-Server, der lokalen **Drone** CI-Instanz und dem lokalen **Pact Broker** gewählt.

4.3.2 Angular-Integration

Die Integration der Contract Tests in die **Angular**-Anwendung stellte eine besondere Herausforderung dar, da die **Angular**-Entwicklungsumgebung und die verwendeten Abhängigkeiten nicht optimal mit den **pact-js**-Bibliotheken harmonierten.

Viele Anleitungen und Beispiele für den **Angular**-Context beziehen sich auf die Plugin-Kombination von **pact-web**, **pact-node** bzw. **karma-pact** [20, 21]. Diese werden allerdings nicht mehr aktiv gepflegt [2, 37]. Deshalb wurde auf die Implementierung mithilfe von **pact-js** gesetzt, welches die Consumer-Tests in TypeScript unterstützt und von der **Pact Foundation** empfohlen wird [36].

Eine direkte Verwendung der Bibliothek in der **Angular**-Umgebung war nicht möglich, da es zu Abhängigkeitskonflikten kam. Um diese bei der Integration von **pact-js** in die **Angular**-Anwendung zu lösen, wurde ein separates Verzeichnis innerhalb des Repositories erstellt. Für die Verwaltung der Abhängigkeiten für **pact-js**, **Jasmine** usw., wurde eine neue **Node.js**-Umgebung im Verzeichnis der Contract Tests eingerichtet. So konnte die Ausführung der Consumer-Tests unabhängig von der **Angular**-Anwendung erfolgen.

4.3.3 Provider-Verifizierung

Bei der Implementierung der Provider-Verifizierungstests wurden die Tests zunächst nicht direkt gegen die reale Implementierung des Providers ausgeführt. Die im Contract definierten Requests wurden an einen zufälligen Port gesendet, der nicht mit dem Port des laufenden Providers übereinstimmte. Dies führte dazu, dass die Tests fehlschlagen, da auf dem angesprochenen Port nicht die erwartete Antwort zurückgegeben wurde. Der Grund für dieses ungewollte Verhalten wurde zunächst nicht bemerkt, da auf dem angesprochenen Port ein anderer undefinierter Server lief und mit seinen HTTP-Responses antwortete. Dadurch verlängerte sich die Dauer des Debugging, bevor die Ursache identifiziert werden konnte.

4.3 Aufgetretene Herausforderungen

Die Lösung bestand darin, den Port des Providers in der Testkonfiguration explizit anzugeben, sodass die Anfragen an den richtigen Port gesendet wurden und die richtigen Antworten zurückkamen. Durch eine Konfiguration in der Testklasse wird der Port des laufenden Providers ermittelt und dieser für die Tests als Zielport spezifiziert. Durch diese Maßnahme wurde sichergestellt, dass alle Tests tatsächlich gegen die beabsichtigte Provider-Implementierung ausgeführt werden und damit die korrekte Verifizierung der vertraglich vereinbarten Schnittstellen erfolgt.

Die Schnittstelleninteraktionen zwischen den Microservices der Anwendung *msg.Sensorik* können nun nach der Implementierung der Contract Tests und dem Lösen der technischen Herausforderungen validiert werden. Durch die Integration der Contract Tests in die CI/CD-Pipeline ist sichergestellt, dass Änderungen an den Services kontinuierlich überprüft werden und die Interoperabilität der Microservices gewährleistet ist. Im nächsten Kapitel werden die Auswirkungen und Vorteile dieser Implementierung bezüglich der Fehlererkennung und Testeffizienz evaluiert.

Kapitel 5

Evaluation der Ergebnisse

Wie in Kapitel 4 beschrieben, konnten die Contract Tests innerhalb der Anwendung *msg.Sensorik* erfolgreich implementiert werden. Bei der konkreten Umsetzung der Architektur ergaben sich jedoch Herausforderungen. Wie in Abschnitt 4.3 dargelegt wurde, konnten diese im Verlauf der vorliegenden Arbeit erfolgreich bewältigt werden.

Nach der prototypischen Implementierung der Contract Tests werden in diesem Kapitel die erzielten Ergebnisse evaluiert. Ziel ist es, zu bewerten, inwieweit die gesetzten Ziele, insbesondere die Reduktion von Integrationsproblemen und die Steigerung der Testeffizienz, durch den Einsatz von Contract Testing erreicht werden konnten.

5.1 Bewertung der Fehlererkennung

Ein zentrales Ziel von Contract Testing ist es, Integrationsfehler zuverlässig und automatisiert zu erkennen. Um die Wirksamkeit der implementierten Tests zu evaluieren, wurde ein gezieltes Testszenario entworfen. Dazu wurden im Rahmen dieser Arbeit absichtlich Änderung mit Kompatibilitätsbruch in den API-Schnittstellen einzelner Services vorgenommen, um zu prüfen, ob diese durch die eingerichteten Contract Tests zuverlässig erkannt werden. Beim Ändern der Schnittstellen wurde darauf geachtet, dass teilweise auch die Contract Tests des jeweiligen Services angepasst wurden, um sowohl unabsichtliche Änderungen als auch praktische Anpassungen einer Schnittstelle durch ein Umsetzungsteam zu simulieren.

Im Folgenden werden exemplarisch drei Arten aller durchgeführten Änderungen dargestellt, die zu Kompatibilitätsproblemen führen können und im Rahmen dieser Arbeit untersucht wurden. Die Änderungen erfolgten schrittweise und wurden nach jeder Anpassung einzeln getestet, um die Reaktion der Contract Tests gezielt zu überprüfen. Zum Einen wurde ein vorhandener API-Pfad beim Consumer umbenannt, sodass Anfragen an den neuen Pfad gesendet wurden. Wie erwartet, schlugen die entsprechenden Contract Tests fehl, da die Anfragen nicht mehr dem definierten Contract entsprachen. Anschließend wurde in einem Testlauf das verwendete HTTP-Verb eines bestehenden Endpunkts auf Provider-Seite geändert, hier von POST auf PUT. In diesem Fall wurden die Requests vom Consumer wie

ursprünglich gesendet, was zu Fehlern in der Provider-Verifizierung führte, da keine übereinstimmende Schnittstelle vorhanden war. Außerdem wurden Änderungen an der erwarteten JSON-Struktur der Nutzdaten vorgenommen durch das Entfernen und Umbenennen einzelner Felder in der Request- oder Response-Payload. Zusätzlich wurden auch die entsprechenden Contract Tests auf der Consumer-Seite angepasst, um die Tätigkeit eines Entwicklers zu simulieren. Hier zeigte sich, dass auch semantische Änderungen innerhalb der Datenformate durch den **can-i-deploy**-Schritt korrekt durch mangelnde Verifizierung des neu erstellten Vertrags erkannt wurden.

Nach Abschluss der Änderungen an einem Service wurde die jeweiligen Test-Pipelines, deren Aufbau bereits in Abschnitt 4.2 beschrieben wurde, automatisiert ausgeführt. Dabei wurden die Änderungen erwartungsgemäß als Fehler erkannt, da die vorgenommenen Änderungen gegen die definierten Contracts verstießen. Entweder schlug die Ausführung der Contract Tests fehl oder die **can-i-deploy**-Überprüfung war nicht erfolgreich, weil die neu erstellten Contracts nicht verifiziert werden konnten. Der Ablauf der Fehlererkennung für Änderungen auf der Consumer-Seite bei der Ausführung der Test-Pipelines ist mit den verschiedenen Varianten der Untersuchung in Abbildung 5.1 dargestellt.

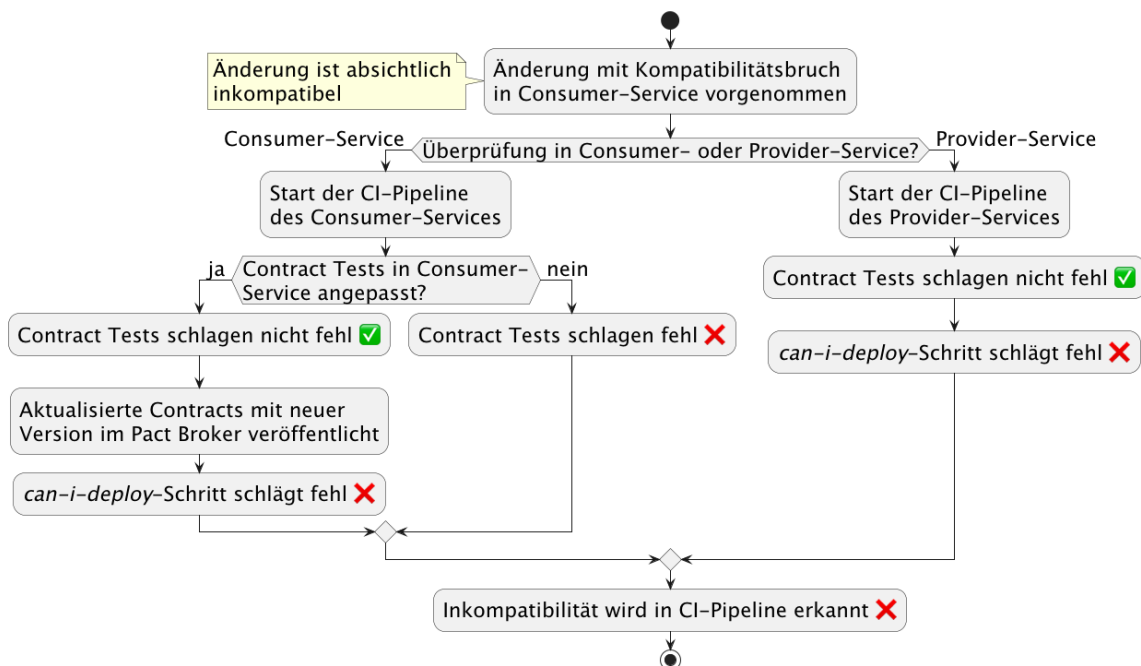


Abbildung 5.1: Ablauf der Fehlererkennung bei der Ausführung der Test-Pipeline

Bei der Ausführung der Test-Pipeline gilt die Fehlererkennung als erfolgreich, wenn die Pipeline aufgrund der enthaltenen Überprüfungen nicht erfolgreich abgeschlossen werden kann. Dieses Fehlschlagen der Pipeline wird dann in der CI/CD-Umgebung protokolliert, dem jeweiligen Entwicklerteam gemeldet und sorgt dadurch dafür, dass weitere Schritte, wie das Deployment, unterbunden werden. So können Integrationsprobleme frühzeitig erkannt

5.2 Bewertung der Testeffizienz

und behoben werden, bevor sie sich in nachfolgenden Phasen des Entwicklungsprozesses negativ auswirken.

Die Ergebnisse dieses kontrollierten Testszenarios zeigen eindeutig, dass alle eingebauten Abweichungen von den vertraglich vereinbarten Schnittstelleninteraktionen durch die Contract Tests identifiziert wurden. Die Fehler wurden jeweils unmittelbar im Rahmen der Consumer- oder Provider-Tests erkannt, wodurch der Integrationsbruch frühzeitig sichtbar wurde und die Interoperabilität zwischen den Services gewährleistet wird. Bei der Analyse der Ergebnisse zeigte sich, dass die Contract Tests sogar in der Lage sind, auch auf der Provider-Seite Fehler zu erkennen, die bei Integrationstests nur durch die Ausführung der Tests auf der Consumer-Seite sichtbar werden. So konnte gezeigt werden, dass Contract Tests Inkompatibilitäten zwischen Consumer und Provider auch bei der unabhängigen Entwicklung von Services auf beiden Seiten zuverlässig erkennen. Dadurch wird die modulare Wartbarkeit und Unabhängigkeit in der Entwicklung von Microservices gefördert.

Ob durch den Einsatz von Contract Testing insgesamt mehr Fehler entdeckt werden als durch herkömmliche Integrationstests, lässt sich im Rahmen dieser Arbeit jedoch nicht abschließend bewerten. Die eingebauten Änderungen mit Kompatibilitätsbrüchen, die in dieser Arbeit testweise durchgeführt wurden, wurden auch durch die Integrationstests erkannt. Eine belastbare Aussage zur Steigerung der Fehlererkennungsrate durch Contract Tests würde eine längerfristige Beobachtung über mehrere Entwicklungszyklen und verschiedene Anwendungen hinweg erfordern, idealerweise mit quantitativen Vergleichsdaten über mehrere Releases oder Sprints hinweg.

Ein Rückgriff auf historische Fehlerdaten ist hierbei nicht zielführend, da frühere API-Implementierungen teils stark von der aktuellen Architektur abweichen. Zudem wären nachträgliche Contract Tests auf historische Implementierungen wenig aussagekräftig, da sie retrospektiv auf bereits bekannten Code angepasst werden würden und somit keine objektive Fehlererkennung ermöglichen.

5.2 Bewertung der Testeffizienz

Neben der Fehlererkennung ist die Effizienz ein entscheidender Aspekt beim Vergleich von Contract Tests mit klassischen Integrationstests. Im Rahmen dieser Arbeit wurden konkrete Messungen zur Ausführungsdauer der Test-Pipelines durchgeführt, um die Auswirkungen auf den Entwicklungsprozess zu bewerten. Die Messungen wurden auf einem Dell Latitude 5521 mit Intel Core i7-11850H und 64 Gigabyte Arbeitsspeicher durchgeführt, auf dem die CI/CD-Umgebung Drone CI mit mehreren Containern für die einzelnen Pipelineschritte lief. Auch wenn absolute Zeitangaben je nach Infrastruktur und Testumgebung variieren

können, ist der relative Vergleich der Messungen zwischen Contract Tests und Integrationstests aufschlussreich.

Das Balkendiagramm in Abbildung 5.2 zeigt jeweils die durchschnittliche Ausführungsdauer der CI/CD-Pipelines für Contract Tests und Integrationstests für die fünf Microservices der Anwendung. Die Zeitangaben basieren auf jeweils zehn Einzelmessungen pro Testtyp und pro Microservice. Zur besseren Vergleichbarkeit wurden die Mittelwerte berechnet und auf ganze Sekunden gerundet, um Schwankungen auszugleichen. Die gesamten Messungen sind in Anhang A.1 detailliert aufgeführt.

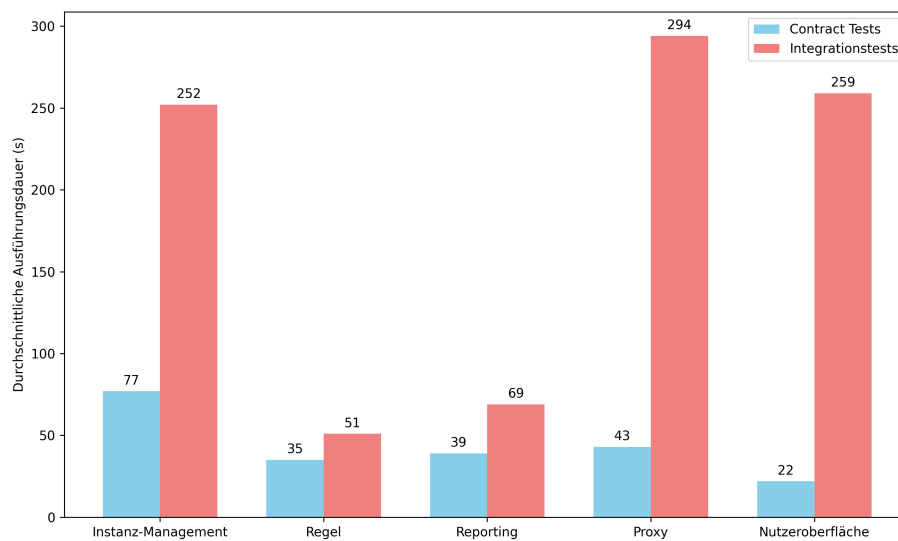


Abbildung 5.2: Vergleich der durchschnittlichen Ausführungsdauer der CI/CD-Pipelines für Contract Tests und Integrationstests

Die Ergebnisse zeigen, dass Contract Tests signifikant schneller ausgeführt werden können als Integrationstests. Der Unterschied ist besonders ausgeprägt bei den Microservices, die als Consumer fungieren, da sie mehr Abhängigkeiten zu anderen Services haben. Die durchschnittliche Ausführungsdauer der Integrationstest-Pipeline ist bis zu 10-mal länger als die der Contract Tests. Dies liegt insbesondere daran, dass Contract Tests unabhängig von anderen Services laufen, also ohne die vollständige Inbetriebnahme aller beteiligten Services und deren Technologiestacks. Dadurch entfällt ein großer Teil des technischen Mehraufwandes, der bei klassischen Integrationstests anfällt, etwa durch das Starten von Containern, Datenbankinitialisierungen oder Netzwerkkonfigurationen. Technische Details und strukturelle Unterschiede dieser zwei Test-Pipelines sind in Anhang A.1 detailliert dargestellt.

Ein wesentlicher Vorteil ergibt sich aus der Möglichkeit, Contract Tests frühzeitig und häufiger im Entwicklungszyklus auszuführen – etwa direkt beim Commit oder beim Prüfen eines Pull Requests [11]. Durch die Unabhängigkeit von anderen Services können Contract Tests sogar lokal auf den Entwicklerrechnern ausgeführt werden, ohne dass die komplexe

5.2 Bewertung der Testeffizienz

Infrastruktur für die Integrationstests aufwändig gestartet werden muss. Während Integrationstests aufgrund der höheren Laufzeit oft erst in späteren CI/CD-Phasen oder erst in dedizierten Testumgebungen stattfinden [43], können Contract Tests bereits während der Entwicklung wertvolles Feedback liefern. Diese frühzeitige Rückmeldung sorgt für eine erhöhte Qualität der Software, da sich früh behobene Fehler nicht auf die nachfolgenden Phasen des Entwicklungsprozesses auswirken [24].

Diese verkürzte Fehlererkennungszeit ist besonders im Kontext der sogenannten *Cost of Failure*-Kurve relevant. Demnach steigen die Kosten zur Behebung eines Fehlers exponentiell mit dem Fortschreiten des Softwareentwicklungsprozesses – von der Anforderungsanalyse über die Implementierung bis hin zum Betrieb [5], wie in Abbildung 5.3 dargestellt. Ein Fehler, der bereits in der Implementierungsphase entdeckt und behoben wird, verursacht einen Bruchteil der Kosten im Vergleich zu einem Fehler, der erst in einer fortgeschrittenen Prozessstufe oder sogar in der Produktionsumgebung auftritt [4].

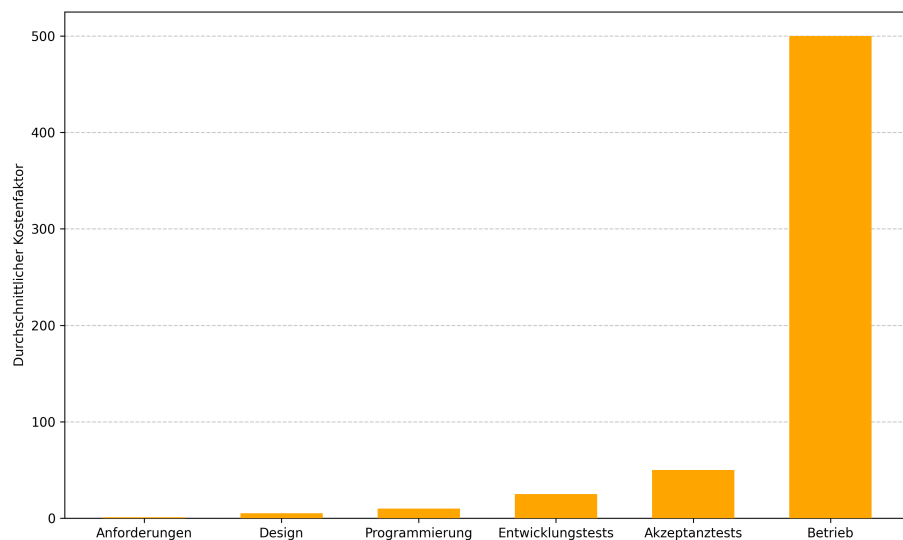


Abbildung 5.3: Cost of Failure-Kurve nach [5]

Diese Faktoren machen deutlich, wie wichtig es ist, bereits in frühen Phasen des Entwicklungsprozesses potenzielle Fehler zu identifizieren und zu beheben. Dieses wird durch Contract Tests begünstigt, da diese eine schnelle und effiziente Überprüfung der Interoperabilität von Schnittstellen erlauben.

Insgesamt lässt sich feststellen, dass der Einsatz von Contract Tests zu einer spürbaren Steigerung der Testeffizienz beitragen kann. Entwickler erhalten schneller Rückmeldung

über API-Kompatibilitätsbrüche, was die Entwicklungszyklen und die Feedback-Schleifen optimiert. Die dadurch gewonnene Agilität kann in komplexeren Anwendungsumgebungen wie z. B. bei Microservice-Architekturen einen entscheidenden Vorteil darstellen. Durch die frühzeitige Detektion von API-Inkompatibilitäten im Rahmen von Contract Tests lassen sich somit nicht nur technische Ressourcen einsparen, sondern auch betriebswirtschaftliche Kosten reduzieren.

Kapitel 6

Fazit und Ausblick

In diesem Kapitel werden die zentralen Ergebnisse der vorliegenden Arbeit zusammengefasst und im Hinblick auf die anfangs formulierten Zielsetzungen reflektiert. Hierbei wurde Contract Testing in eine reale Microservice-Anwendung prototypisch integriert. Anhand des Ergebnisses wird bewertet, inwiefern die angestrebten Verbesserungen hinsichtlich Testbarkeit, Effizienz und Fehlertoleranz erreicht wurden.

Darüber hinaus wird ein Ausblick auf mögliche zukünftige Forschungsvorhaben gegeben. Diese könnten sich mit der Weiterentwicklung, verschiedenen Anwendungen oder alternativen Varianten von Contract Testing befassen. Außerdem ergeben sich Potenziale und offene Fragestellungen, die über den im Rahmen dieser Arbeit untersuchten Anwendungsfall hinausgehen und zur Weiterentwicklung der Methodik beitragen können.

6.1 Fazit

Ziel dieser Arbeit war die prototypische Integration von Contract Testing in eine bestehende Microservice-Anwendung. Zu diesem Zweck wurden zunächst mehrere etablierte Frameworks systematisch analysiert. Auf Basis definierter Bewertungskriterien erfolgte eine vergleichende Bewertung der Werkzeuge, die sowohl qualitative als auch quantitative Aspekte berücksichtigte. Durch die Auswertung der unternehmensspezifisch gewichteten Bewertungskriterien konnte eine Rangfolge der untersuchten Frameworks hinsichtlich ihrer Eignung für das vorliegende Anwendungsszenario abgeleitet werden. Auf dieser Grundlage fiel die Entscheidung zugunsten des Frameworks **Pact**, das im weiteren Verlauf prototypisch in die reale Anwendung *msg.Sensorik* integriert wurde.

Die praktische Umsetzung der Contract Tests konnte – trotz einiger technischer Herausforderungen – im Rahmen dieser Arbeit erfolgreich abgeschlossen werden. Es zeigte sich, dass **Pact** eine gut integrierbare Lösung darstellt, die sich sowohl in bestehende CI/CD-Pipelines einfügt als auch frühzeitiges Feedback bei API-Änderungen ermöglicht.

Die Evaluation der Implementierung hat gezeigt, dass Contract Testing eine wirksame Methode zur Qualitätssicherung von Schnittstellen insbesondere in Microservice-Architekturen

darstellt. Durch die automatisierte Prüfung der vertraglich definierten Interaktionen zwischen Consumer und Provider lassen sich Schnittstelleninkompatibilitäten zuverlässig identifizieren. Die im Rahmen dieser Arbeit durchgeführten Tests bestätigten, dass eingeführte API-Veränderungen vollständig durch die Contract Tests erkannt wurden.

Zudem konnte eine messbare Verbesserung der Testeffizienz und somit eine frühzeitige Erkennung von Integrationsproblemen im Entwicklungsprozess festgestellt werden. Contract Tests benötigen keine aufwändige Infrastruktur und lassen sich schneller ausführen als klassische Integrationstests. Dies erlaubt häufigere Ausführungen und schnellere Rückmeldungen im Entwicklungsprozess, was zu verkürzten Feedback-Zyklen und einer höheren Reaktionsgeschwindigkeit führt. Somit wird die Qualität der Software erhöht. Außerdem werden durch Contract Tests die Kosten für Fehlerbehebungen oder Ausfälle von Anwendungen gesenkt, da frühzeitige Fehlerbehebungen geringere Kosten verursachen.

Eine Verbesserung der Fehlererkennungsrate konnte im Rahmen dieser Arbeit nicht quantitativ belegt werden, da eine umfassende Analyse über mehrere Releases und Sprints hinweg nicht durchgeführt werden konnte.

Insgesamt bestätigt die durchgeführte Evaluation, dass Contract Testing einen relevanten Beitrag zur frühzeitigen effizienten Fehlererkennung und eine wichtige Ergänzung der klassischen Testverfahren darstellt – besonders in verteilten, dynamischen und schnelllebigem Systemlandschaften, wie sie typischerweise in Microservice-Architekturen anzutreffen sind. Contract Testing sorgt für die Validierung der Schnittstelleninteraktionen, während Integrationstests das korrekte Zusammenspiel der Services und deren Technologiestacks gewährleisten. So können die Stärken der jeweiligen Testarten genutzt werden. Beide Testarten ergänzen sich gegenseitig, was zu einer robusteren Teststrategie führt.

Für zukünftige Projekte, die Contract Testing in die jeweilige Anwendungsumgebung integrieren möchten, lassen sich auf Basis der Erkenntnisse dieser Arbeit mehrere Handlungsempfehlungen ableiten. Die Einführung sollte möglichst frühzeitig erfolgen, damit sich das volle Potenzial zur Fehlerprävention entfalten kann. Der Einsatz eines zentralen Contract Brokers ist empfehlenswert, um die Verwaltung und den Austausch der Verträge zwischen Consumer und Provider zu erleichtern. Die Integration in CI/CD-Pipelines sollte von Anfang an berücksichtigt werden, um die Tests automatisiert und regelmäßig ausführen zu können. Die Auswahl des richtigen Frameworks sollte auf Basis der spezifischen Anforderungen und Gegebenheiten des Projekts erfolgen, wobei die in dieser Arbeit definierten Bewertungskriterien als Orientierung dienen können. Allgemein hat sich **Pact** durch die flexible Einsetzbarkeit, praktische Contract-Verwaltung durch den **Pact Broker** und die aktive Gemeinschaft als eine robuste und sinnvolle Lösung erwiesen, die Integration von Contract Testing zu unterstützen.

6.2 Ausblick

Die in dieser Arbeit gewonnenen Erkenntnisse bilden eine solide Grundlage für die Bewertung und Einführung von Contract Testing in realen Microservice-Umgebungen. Im Zuge der Erstellung dieser Arbeit haben sich zudem mehrere interessante Themengebiete ergeben, die als potenzielle Inhalte zukünftiger Forschungsprojekte dienen können.

Ein naheliegender nächster Schritt wäre die langfristige Begleitung eines Projekts mit aktiv genutzten Contract Tests über mehrere Iterationen hinweg. Eine solche Untersuchung könnte beispielsweise die Auswirkungen auf Fehlerraten, Wartungsaufwand und Release-Zyklen systematisch erfassen und quantifizieren. Insbesondere die Analyse von Daten im Vergleich zu Entwicklungsprozessen ohne Einsatz von Contract Testing wäre geeignet, den tatsächlichen Einfluss auf die Qualität und Stabilität der Systemlandschaft weiter zu präzisieren.

Ein weiteres mögliches Untersuchungsfeld ist das Provider-driven Contract Testing. Im Gegensatz zum Ansatz, der vom Consumer ausgeht, werden die Verträge von beiden Seiten definiert und veröffentlicht. Dies ist insbesondere dann sinnvoll, wenn die Provider-Seite eine Drittanbieter-API ist und die Consumer-Seite nicht die Möglichkeit hat, diese API zu beeinflussen. Zukünftige Arbeiten könnten untersuchen, in welchen Szenarien dieser Ansatz vorteilhaft ist und wie er sich technisch und organisatorisch umsetzen lässt.

Nicht zuletzt könnte zukünftig untersucht werden, wie sich Contract Testing auf die Tätigkeiten von Entwicklern und anderen Projektbeteiligten bzgl. Arbeitsweise, Zeitaufwand und Kommunikation zwischen Projektteams auswirkt. Dazu könnten qualitative Interviews durchgeführt werden, um Erfahrungen und Wahrnehmungen der Beteiligten zu erfassen.

Zusammenfassend lässt sich festhalten, dass Contract Testing nicht nur ein technisches Werkzeug zur Schnittstellenprüfung ist, sondern ein umfassendes Konzept zur Qualitätssicherung von Schnittstelleninteraktionen darstellt. Die Ergebnisse dieser Arbeit bieten hierfür eine fundierte Ausgangsbasis, auf der zukünftige Entwicklungen aufbauen können.

Anhang A

Weiterführende Informationen

A.1 Komplexitätsunterschied der untersuchten Testpipelines

In Abbildung A.1 ist die Pipeline für die Integrationstests des Instanz-Management-Services dargestellt. Die Integrationstests benötigen alle abhängigen Services und deren Technologiestack, um korrekt zu funktionieren und die Interaktion zwischen den Services zu testen. Dies macht sie komplexer in Bezug auf Setup-Zeit und Wartbarkeit. Jeder Dienst muss gestartet und korrekt konfiguriert sein, bevor Tests sinnvoll durchgeführt werden können.

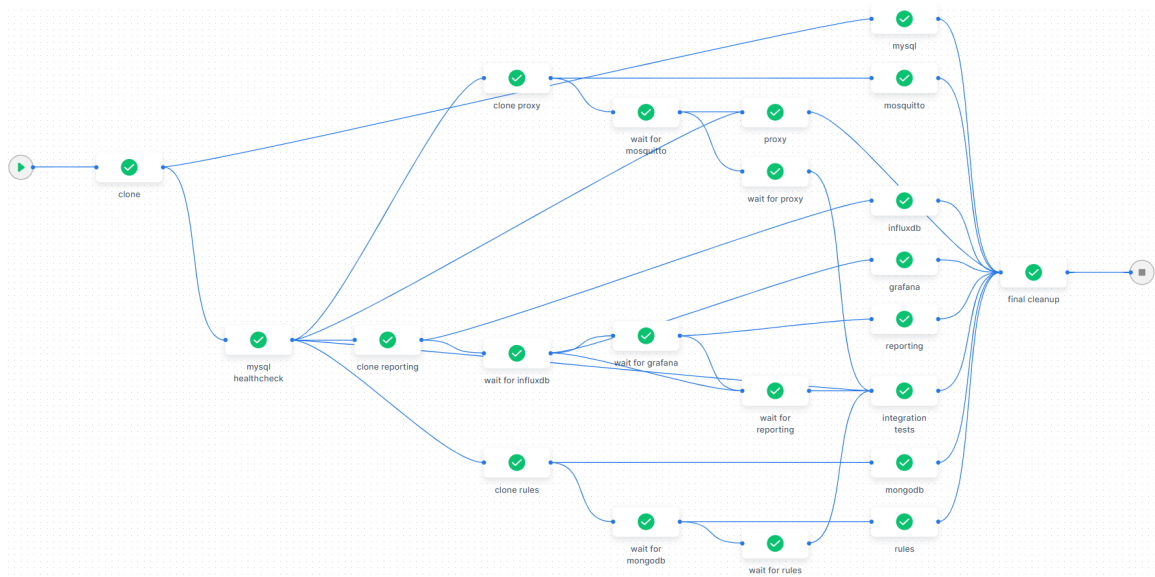


Abbildung A.1: Ablauf der Integrationstest-Pipeline des Instanz-Management-Services

Im Vergleich dazu ist die in Abbildung A.2 dargestellte Contract-Test-Pipeline deutlich schlanker. Sie testet gezielt die API-Interaktionen zwischen Services anhand der definierten Contracts. Dadurch entfällt das Deployment sämtlicher Dienste, was zu kürzeren Ausführungszeiten und geringerer Komplexität führt. Die Wartung ist ebenfalls einfacher, da nur die jeweiligen Contracts und nicht die eigentlichen Komponenten berücksichtigt werden.

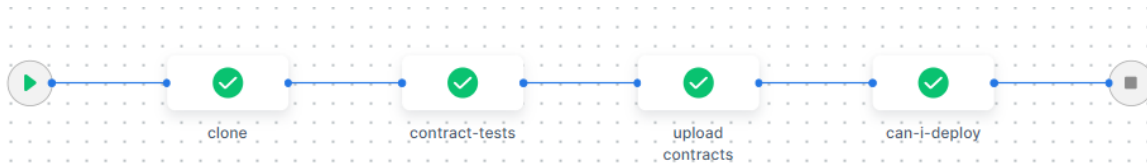


Abbildung A.2: Ablauf der Contract-Test-Pipeline

Die Integrationstest-Pipeline ist in der Regel komplexer, da sie alle abhängigen Services und deren Bestandteile abbildet. Dies erhöht sowohl die Pipelinedauer als auch den Wartungsaufwand. Die Contract-Test-Pipeline hingegen hat weniger Abhängigkeiten und somit auch weniger Pipelineschritte. Dadurch ist sie schneller und eignet sich für häufige Ausführungen und für frühzeitige Fehlererkennung.

A.2 Tatsächliche Messwerte der Testpipeline-Ausführungszeiten

Die tatsächlichen Messwerte der Testpipeline-Ausführungszeiten sind für die Contract-Test-Pipeline in Tabelle A.1 und für die Integrationstest-Pipeline in Tabelle A.2 detailliert aufgelistet.

Tabelle A.1: Contract-Test-Messwerte pro Service

Service	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10
Instanz-Management	72	81	77	73	74	79	73	80	82	82
Regel	44	41	34	31	30	33	39	31	32	31
Reporting	44	43	38	38	37	38	38	40	34	38
Proxy	48	38	39	47	42	42	41	43	45	44
Nutzeroberfläche	22	25	22	23	24	20	21	23	21	22

Tabelle A.2: Integrationstest-Messwerte pro Service

Service	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10
Instanz-Management	258	260	255	248	250	244	250	259	249	243
Regel	54	48	49	49	46	50	58	47	51	56
Reporting	84	74	67	67	63	68	69	69	67	65
Proxy	284	274	291	288	289	304	293	301	300	314
Nutzeroberfläche	234	285	266	254	251	257	262	267	249	261

Abbildungsverzeichnis

2.1	Testpyramide nach [24]	7
2.2	Ablauf eines Contract Tests auf der Consumer-Seite beim CDCT nach [11]	11
2.3	Ablauf eines Contract Tests auf der Provider-Seite beim CDCT nach [11]	12
2.4	Ablauf eines Contract Tests beim PDCT nach [11]	14
4.1	Abhängigkeitsgraph der Microservices	38
5.1	Ablauf der Fehlererkennung bei der Ausführung der Test-Pipeline	44
5.2	Vergleich der durchschnittlichen Ausführungsdauer der CI/CD-Pipelines für Con- tract Tests und Integrationstests	46
5.3	Cost of Failure-Kurve nach [5]	47
A.1	Ablauf der Integrationstest-Pipeline des Instanz-Management-Services	53
A.2	Ablauf der Contract-Test-Pipeline	54

Tabellenverzeichnis

3.1 Vergleich der Contract-Testing-Frameworks	30
4.1 Consumer/Provider-Matrix der Microservices	33
A.1 Contract-Test-Messwerte pro Service	54
A.2 Integrationstest-Messwerte pro Service	54

Verzeichnis der Listings

4.1 Beispiel für einen Pact-Contract	34
--	----

Glossar

Angular Ein weit verbreitetes Framework für die Entwicklung von Webanwendungen, das auf TypeScript basiert und eine komponentenbasierte Architektur bietet. Angular wird häufig für die Entwicklung von Single-Page-Anwendungen verwendet. 32, 34, 36, 40

Build Der Prozess, bei dem der Quellcode in ein ausführbares Format umgewandelt wird. Dies kann das Kompilieren von Code, das Zusammenstellen von Ressourcen und das Erstellen von Installationspaketen umfassen. 22, 25, 26, 27, 31, 32

Deployment Der Prozess, bei dem eine Anwendung oder ein System in einer Produktionsumgebung bereitgestellt wird. Dies umfasst die Installation, Konfiguration und Aktivierung der Software auf den Zielsystemen. 6, 8, 13, 22, 24, 44, 53

DevOps Eine Kombination aus Entwicklung (Dev) und Betrieb (Ops), die darauf abzielt, die Zusammenarbeit zwischen Softwareentwicklern und IT-Betriebsteams zu verbessern. DevOps fördert Automatisierung, kontinuierliche Integration und kontinuierliche Bereitstellung. 8, 9

Docker Eine Plattform zur Automatisierung der Bereitstellung von Anwendungen in Containern, die eine konsistente und isolierte Ausführungsumgebung bieten. Docker ermöglicht es Entwicklern, Anwendungen und ihre Abhängigkeiten in Containern zu verpacken und zu verteilen. 32, 33

Gradle Ein Build-Management-Tool, das eine flexible Konfiguration von Build-Prozessen ermöglicht. Gradle wird häufig in Java-Projekten verwendet, bietet aber auch Unterstützung für andere Programmiersprachen. 22, 26, 27, 28, 29

GraphQL Eine Abfragesprache für APIs, die es Clients ermöglicht, genau die Daten abzufragen, die sie benötigen. GraphQL bietet eine flexible und effiziente Möglichkeit, Daten von Servern abzurufen und zu manipulieren. 20, 23, 25, 27

gRPC Ein modernes, leistungsfähiges Remote Procedure Call (RPC)-Framework, das auf HTTP basiert und eine effiziente Kommunikation zwischen Diensten ermöglicht. gRPC unterstützt mehrere Programmiersprachen und bietet Funktionen wie Streaming und bidirektionale Kommunikation. 25

- Junit5** Eine weit verbreitete Testbibliothek für Java, die eine einfache und flexible Möglichkeit bietet, Tests zu schreiben und auszuführen. 37
- Kafka** Eine verteilte Datenstrom-Plattform, die es ermöglicht, große Mengen an Daten in Echtzeit zu verarbeiten und zu speichern. Kafka wird häufig für Event-Streaming und Messaging in Microservice-Architekturen verwendet. 23
- Maven** Ein Build-Management-Tool für Java-Projekte, das die Verwaltung von Abhängigkeiten, das Erstellen von Artefakten und die Ausführung von Tests automatisiert. Maven ist ein wichtiger Bestandteil des Java-Ökosystems. 22, 26, 27, 28, 29, 32, 37
- Mock** Ein Testdoppel, das das Verhalten eines echten Objekts simuliert. 11, 12, 20, 34
- NoSQL** Eine Klasse von Datenbanken, die nicht auf dem relationalen Modell basieren und für unstrukturierte oder semi-strukturierte Daten geeignet sind. NoSQL-Datenbanken bieten Flexibilität und Skalierbarkeit für moderne Anwendungen. 32
- OpenAPI** Eine Spezifikation für REST-APIs, die es Entwicklern ermöglicht, APIs zu beschreiben, zu erstellen und zu dokumentieren. OpenAPI verwendet eine standardisierte Syntax, um API-Endpunkte, Parameter und Datenformate zu definieren. 18, 26, 27
- Pipeline** Eine Abfolge von automatisierten Schritten, die zur Erstellung, Prüfung und Bereitstellung von Software verwendet werden. Pipelines sind ein zentraler Bestandteil von CI/CD-Workflows. 1, 2, 9, 13, 14, 16, 22, 27, 28, 29, 31, 33, 38, 39, 40, 41, 44, 45, 46, 49, 50, 53, 54, 55
- Spring** Ein weit verbreitetes Framework für die Entwicklung von Java-Anwendungen, das eine Vielzahl von Modulen und Funktionen bietet, um die Entwicklung zu erleichtern. Spring ist besonders bekannt für seine Unterstützung von API-Entwicklung und Microservices. 18, 25, 26, 32, 37
- Stakeholder** Eine Person oder Gruppe, die ein Interesse an einem Projekt hat und von dessen Ergebnissen betroffen ist. Stakeholder können Kunden, Benutzer, Entwickler, Manager und andere Beteiligte sein. 7
- WebSocket** Ein Protokoll, das eine bidirektionale Kommunikation zwischen Client und Server über eine einzelne, dauerhafte Verbindung ermöglicht. WebSocket wird häufig für Echtzeitanwendungen wie Chat-Apps oder Live-Daten-Updates verwendet. 23

Akronyme

API Application Programming Interface. 5, 6, 9, 10, 11, 12, 13, 14, 16, 18, 19, 22, 26, 27, 28, 32, 33, 34, 36, 43, 45, 48, 49, 50, 51, 53, 61, 62

BSD Berkeley Software Distribution. 29

CD Continuous Delivery. 1, 2, 3, 9, 14, 16, 20, 22, 23, 25, 26, 27, 28, 29, 38, 39, 40, 41, 44, 45, 46, 47, 49, 50, 55, 62

CDCT Consumer-Driven Contract Testing. 11, 12, 13, 17, 18, 33, 55

CI Continuous Integration. 1, 2, 3, 8, 9, 14, 16, 20, 22, 23, 25, 26, 27, 28, 29, 38, 39, 40, 41, 44, 45, 46, 47, 49, 50, 55, 62

CLI Command Line Interface. 23, 25, 26, 27, 28, 37

DSL Domain-Specific Language. 27, 28

HTTP Hypertext Transfer Protocol. 10, 23, 25, 27, 34, 36, 40, 43

JSON JavaScript Object Notation. 10, 23, 27, 34, 36, 44

MIT Massachusetts Institute of Technology. 24, 26, 28

MQTT Message Queuing Telemetry Transport. 23

npm Node Package Manager. 22, 29

PDCT Provider-Driven Contract Testing. 11, 13, 14, 55

REST Representational State Transfer. 20, 23, 25, 26, 27, 28, 32, 62

RPC Remote Procedure Call. 20, 23

SOAP Simple Object Access Protocol. 23, 26, 27

URL Uniform Resource Locator. 37

XML eXtensible Markup Language. 27

YAML YAML Ain't Markup Language (ehemals Yet Another Markup Language). 25

Literatur

- [1] `@contract-case/contract-case-jest`, <https://www.npmjs.com/package/@contract-case/contract-case-jest>, Mai 2025. (besucht am 07.06.2025).
- [2] `@pact-foundation/pact-web`, <https://www.npmjs.com/package/@pact-foundation/pact-web>, Juni 2022. (besucht am 02.06.2025).
- [3] *Apache License, Version 2.0*, <https://www.apache.org/licenses/LICENSE-2.0.html>. (besucht am 03.06.2025).
- [4] B. W. Boehm, „Software Engineering Economics,“ *IEEE Transactions on Software Engineering*, Jg. SE-10, Nr. 1, S. 4–21, Jan. 1984, ISSN: 1939-3520. DOI: 10.1109/TSE.1984.5010193.
- [5] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ [u.a.]: Prentice-Hall, 1981, ISBN: 978-0-13-822122-5.
- [6] *BSD 3-Clause "New" or "Revised" License / Software Package Data Exchange (SPDX)*, <https://spdx.org/licenses/BSD-3-Clause.html>. (besucht am 03.06.2025).
- [7] *Case-Contract-Testing/Contract-Case*, ContractCase Contract Testing, Mai 2025. (besucht am 17.05.2025).
- [8] T. Clemson, *Testing Strategies in a Microservice Architecture*, <https://martinfowler.com/articles/microservice-testing/>, Nov. 2014. (besucht am 19.03.2025).
- [9] *Contract Driven Development: Microservices done right without integration pain*, <https://specmatic.io/>. (besucht am 17.05.2025).
- [10] *ContractCase Contract Testing Suite / ContractCase Contract Testing*, <https://case.contract-testing.io/docs/intro/>. (besucht am 17.05.2025).
- [11] M. Cruz und L. Prescott, *Contract Testing in Action*. Shelter Island, NY: Manning Publications Co., Jan. 2025, ISBN: 978-1-63343-724-1.
- [12] I.-C. Donca, O. P. Stan, M. Misaros, D. Gota und L. Miclea, „Method for Continuous Integration and Deployment Using a Pipeline Generator for Agile Software Projects,“ *Sensors (Basel, Switzerland)*, Jg. 22, Nr. 12, S. 4637, Juni 2022, ISSN: 1424-8220. DOI: 10.3390/s22124637.
- [13] H. Dowalil, *Grundlagen Des Modularen Softwareentwurfs*. München: Hanser, ISBN: 978-3-446-45600-6.

- [14] *Drone CI / CD / Drone*, <https://docs.drone.io/>. (besucht am 02.06.2025).
- [15] P. M. Duvall, S. Matyas und A. Glover, *Continuous Integration: Improving Software Quality and Reducing Risk*. Boston, MA: Pearson Education, Inc., Juni 2007, ISBN: 978-0-321-33638-5.
- [16] M. Fowler, *Consumer-Driven Contracts: A Service Evolution Pattern*, <https://martinfowler.com/articles/consumerDrivenContracts.html>. (besucht am 13.04.2025).
- [17] FreeWheel Biz-UI Team, *Cloud-Native Application Architecture: Microservice Development Best Practice*. Singapore: Springer Nature, 2024, ISBN: 978-981-19-9781-5 978-981-19-9782-2. DOI: 10.1007/978-981-19-9782-2.
- [18] J. Haro, *Microservice APIs*. Shelter Island, NY: Manning Publications Co., Feb. 2023, ISBN: 978-1-61729-841-7.
- [19] A. Hemon, B. Fitzgerald, B. Lyonnet und F. Rowe, „Innovative Practices for Knowledge Sharing in Large-Scale DevOps,“ *IEEE Software*, Jg. 37, Nr. 3, S. 30–37, Mai 2020, ISSN: 1937-4194. DOI: 10.1109/MS.2019.2958900.
- [20] R. Hendricksen, *Consumer-Driven Contract Testing with Pact for Angular and Spring Boot*, Nov. 2020. (besucht am 02.06.2025).
- [21] T. Hombergs, *Creating a Consumer-Driven Contract with Angular and Pact*, <https://reflectoring.io/consumer-driven-contracts-with-angular-and-pact/>, Dez. 2017. (besucht am 02.06.2025).
- [22] J. Humble, *Continuous Delivery* (The Addison-Wesley Signature Series), 6. print. Upper Saddle River, NJ [u.a.]: Addison-Wesley, 2012, ISBN: 978-0-321-60191-9.
- [23] *Karatelabs/Karate*, Karate Labs, Mai 2025. (besucht am 17.05.2025).
- [24] T. Linz und A. Spillner, *Basiswissen Softwaretest*, 7. Aufl. Heidelberg: dpunkt.verlag GmbH, Apr. 2024, ISBN: 978-3-98889-005-4.
- [25] *Microservices and API testing framework*, <https://pact.io/>. (besucht am 17.05.2025).
- [26] *MIT License*, <https://mit-license.org/>. (besucht am 01.06.2025).
- [27] N. B. Moe, D. Cruzes, T. Dybå und E. Mikkelsen, „Continuous Software Testing in a Globally Distributed Project,“ in *2015 IEEE 10th International Conference on Global Software Engineering*, Juli 2015, S. 130–134. DOI: 10.1109/ICGSE.2015.24.
- [28] A. Morgan, *Testing Microservices with Consumer-Driven Contracts*. [Place of publication not identified]: O’Reilly Media.
- [29] R. Moskowitz, D. Karrenberg, Y. Rekhter, E. Lear und G. J. de Groot, „Address Allocation for Private Internets,“ Internet Engineering Task Force, Request for Comments RFC 1918, Feb. 1996, S. 9. DOI: 10.17487/RFC1918. (besucht am 02.06.2025).

- [30] S. Newman, *Monolith to Microservices*. Sebastopol, CA: O'Reilly Media, Inc., Nov. 2019, ISBN: 978-1-4920-4783-4.
- [31] S. Newman, *Building Microservices*, 2. Aufl. Sebastopol, CA: O'Reilly Media, Inc., Aug. 2021, ISBN: 978-1-4920-3401-8.
- [32] *Open Source Solution for API Testing*, <https://www.karatelabs.io/>. (besucht am 01.06.2025).
- [33] *OpenAPI Documentation*, <https://learn.openapis.org/>. (besucht am 03.06.2025).
- [34] *Origin of Karate I Karate Labs*, <https://www.karatelabs.io/company>. (besucht am 17.05.2025).
- [35] *Pact Foundation*. (besucht am 03.06.2025).
- [36] Pact Foundation, *Pact Docs*, <https://docs.pact.io/>, Aug. 2022. (besucht am 17.05.2025).
- [37] *Pact-foundation/karma-pact: Pact Framework Plugin for Karma*, <https://github.com/pact-foundation/karma-pact>. (besucht am 02.06.2025).
- [38] PactFlow, *Bi-Directional Contract Testing Guide | PactFlow Documentation*, <https://docs.pactflow.io/docs/bi-directional-contract-testing>, Documentantation, Juli 2023. (besucht am 17.04.2025).
- [39] *Pactflow Contract Testing Platform*, <https://pactflow.io/>. (besucht am 03.06.2025).
- [40] U. Post, *Besser Coden: Best Practices Für Clean Code*, 2., aktualisierte und erweiterte Auflage. Bonn: Rheinwerk Computing, 2021, ISBN: 978-3-8362-8492-9.
- [41] C. Richardson, *Microservices Patterns*. Shelter Island, NY: Manning Publications Co., Nov. 2018, ISBN: 978-1-61729-454-9.
- [42] M. Shahin, M. Ali Babar und L. Zhu, „Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices,“ *IEEE Access*, Jg. 5, S. 3909–3943, 2017, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2017.2685629.
- [43] Y. Singh, *Software Testing*. Cambridge: Cambridge University Press, 2011, ISBN: 978-1-107-01296-7. DOI: 10.1017/CB09781139196185.
- [44] *SonarQube Server 2025.3 | Documentation*, <https://docs.sonarsource.com/sonarqube-server/latest/>. (besucht am 07.06.2025).
- [45] *Specmatic Documentation | A Contract Driven Development Tool*, <https://docs.specmatic.io/>. (besucht am 17.05.2025).
- [46] *Spring Cloud Contract*, <https://spring.io/projects/spring-cloud-contract>. (besucht am 17.05.2025).

- [47] *Spring Cloud Contract 1.0.0.RELEASE is available*,
<https://spring.io/blog/2016/09/23/spring-cloud-contract-1-0-0-release-is-available>.
 (besucht am 17.05.2025).
- [48] *Spring Cloud Contract Reference Documentation :: Spring Cloud Contract*,
<https://docs.spring.io/spring-cloud-contract/reference/>. (besucht am 01.06.2025).
- [49] *Spring-Cloud/Spring-Cloud-Contract*, Spring Cloud, Juni 2025. (besucht am 03.06.2025).
- [50] H. Van Merode, *Continuous Integration (CI) and Continuous Delivery (CD): A Practical Guide to Designing and Developing Pipelines*. Berkeley, CA: Apress, 2023, ISBN: 978-1-4842-9227-3 978-1-4842-9228-0. DOI: 10.1007/978-1-4842-9228-0.
- [51] S. Vijayakumaran, *DevOps: Frameworks, Techniques, and Tools*. Place of publication not identified: Rheinwerk Publishing Inc, 2025, ISBN: 978-1-4932-2670-2.
- [52] M. Waseem, P. Liang und M. Shahin, „A Systematic Mapping Study on Microservices Architecture in DevOps,“ *Journal of Systems and Software*, Jg. 170, S. 110 798, Dez. 2020, ISSN: 0164-1212. DOI: 10.1016/j.jss.2020.110798.
- [53] M. Waseem, P. Liang, M. Shahin, A. Di Salle und G. Márquez, „Design, Monitoring, and Testing of Microservices Systems: The Practitioners’ Perspective,“ *Journal of Systems and Software*, Jg. 182, S. 111 061, Dez. 2021, ISSN: 0164-1212. DOI: 10.1016/j.jss.2021.111061.
- [54] *Znsio/Specmatic*, Znsio, Mai 2025. (besucht am 17.05.2025).